

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

**DIGITAL NOTES
ON
MACHINE LEARNING
R20A0518**

**B.TECH III YEAR–II SEM
(R20) REGULATION
(2023-24)**



Prepared by padmaja

MALLAREDDY COLLEGE OF ENGINEERING & TECHNOLOGY
(Autonomous Institution–UGC, Govt. of India)
Recognized under 2(f) and 12(B) of UGC ACT 1956
(Affiliated to JNTUH, Hyderabad, Approved by AICTE–Accredited by NBA & NAAC–‘A’ Grade–ISO 9001:2015 Certified)
Maisammaguda, Dhulapally (Post Via. Hakimpet), Secunderabad–500100, Telangana State, India

(R20A0518) MACHINE LEARNING

UNIT – I

Introduction: Introduction to Machine learning, Supervised learning, Unsupervised learning, Reinforcement learning. Deep learning.

Feature Selection: Filter, Wrapper, Embedded methods.

Feature Normalization:- min-max normalization, z-score normalization, and constant factor normalization

Introduction to Dimensionality Reduction: Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA)

UNIT – II

Supervised Learning – I (Regression/Classification)

Regression models: Simple Linear Regression, multiple linear Regression. Cost Function, Gradient Descent, Performance Metrics: Mean Absolute Error(MAE), Mean Squared Error(MSE) R-Squared error, Adjusted R Square.

Classification models: Decision Trees-ID3, CART, Naive Bayes, K-Nearest-Neighbours (KNN), Logistic Regression, Multinomial Logistic Regression

Support Vector Machines (SVM) - Nonlinearity and Kernel Methods

UNIT – III

Supervised Learning – II (Neural Networks)

Neural Network Representation – Problems – Perceptrons, Activation Functions, Artificial Neural Networks (ANN) , Back Propagation Algorithm.

Convolutional Neural Networks - Convolution and Pooling layers, , Recurrent Neural Networks (RNN).

Classification Metrics: Confusion matrix, Precision, Recall, Accuracy, F-Score, ROC curves

UNIT - IV

Model Validation in Classification : Cross Validation - Holdout Method, K-Fold, Stratified KFold, Leave-One-Out Cross Validation. Bias-Variance tradeoff, Regularization , Overfitting, Underfitting. Ensemble Methods: Boosting, Bagging, Random Forest.

UNIT – V

Unsupervised Learning : Clustering-K-means, K-Modes, K-Prototypes, Gaussian Mixture Models, Expectation-Maximization.

Reinforcement Learning: Exploration and exploitation trade-offs, non-associative learning, Markov decision processes, Q-learning.

TEXT BOOKS:

1. Machine Learning –Saikat Dutt, Subramanian Chandramouli, Amit Kumar Das, Pearson .
2. Foundations of Machine Learning, Mehryar Mohri, Afshin Rostamizadeh, Ameet Talwalkar, MIT Press.
3. Kevin Murphy, Machine Learning: A Probabilistic Perspective, MIT Press,2012

REFERENCE BOOKS:

1. Trevor Hastie, Robert Tibshirani, Jerome Friedman, The Elements of Statistical Learning, Springer2009
2. Christopher Bishop, Pattern Recognition and Machine Learning, Springer,2007.
3. Machine Learning Yearning, Andrew Ng.
4. Data Mining–Concepts and Techniques -Jiawei Han and Micheline Kamber,Morgan Kaufmann

INDEX

UNIT- NO	TOPIC	PAGE NO
I	Introduction to Machine learning	1
	Supervised learning, Unsupervised learning	3
	Reinforcement learning, Deep learning	3
	Feature Selection: Filter, Wrapper , Embedded methods	4
	Feature Normalization: min-max normalization, z-score normalization, constant factor normalization	4
	Introduction to Dimensionality Reduction: Principal Component Analysis(PCA)	6
	Linear Discriminant Analysis(LDA)	7
II	Supervised Learning – I Introduction	9
	Regression models: Simple Linear Regression, multiple linear Regression	16
	Cost Function, Gradient Descent	16
	Performance Metrics: Mean Absolute Error(MAE)	17
	Mean Squared Error(MSE) R-Squared error, Adjusted R Square.	18
	Classification models: Decision Trees-ID3,CART	20
	Naive Bayes, K-Nearest-Neighbours (KNN)	26
	Multinomial Logistic Regression Support Vector Machines (SVM)	35
	Nonlinearity and Kernel Methods	43
III	Supervised Learning – II (Neural Networks) Introduction	46
	Neural Network Representation – Problems – Perceptrons , Activation Functions	46
	Artificial Neural Networks (ANN),Back Propagation Algorithm.	48
	Convolutional Neural Networks - Convolution and Pooling layers	52
	Recurrent Neural Networks (RNN).	54
	Classification Metrics: Confusion matrix	56

	Recall, Accuracy, F-Score, ROC curves	57
IV	Model Validation in Classification : Cross Validation - Holdout Method, K-Fold	59
	Stratified K-Fold, Leave-One-Out Cross Validation.	62
	Bias-Variance tradeoff, Regularization	63
	Overfitting, Underfitting	63
	Ensemble Methods: Boosting, Bagging, Random Forest.	64
V	Unsupervised Learning: Clustering-K-means, K-Modes	69
	K-Prototypes, Gaussian Mixture Models	73
	Expectation-Maximization.	74
	Reinforcement Learning: Exploration and exploitation trade-offs	75
	Non-associative learning	77
	Markov decision processes, Q-learning.	79

UNIT-1

Introduction: Introduction to Machine learning , Supervised learning, Unsupervised learning, Reinforcement learning. Deep learning.

Feature Selection: Filter, Wrapper , Embedded methods.

Feature Normalization:- min-max normalization, z-score normalization, and constant factor normalization

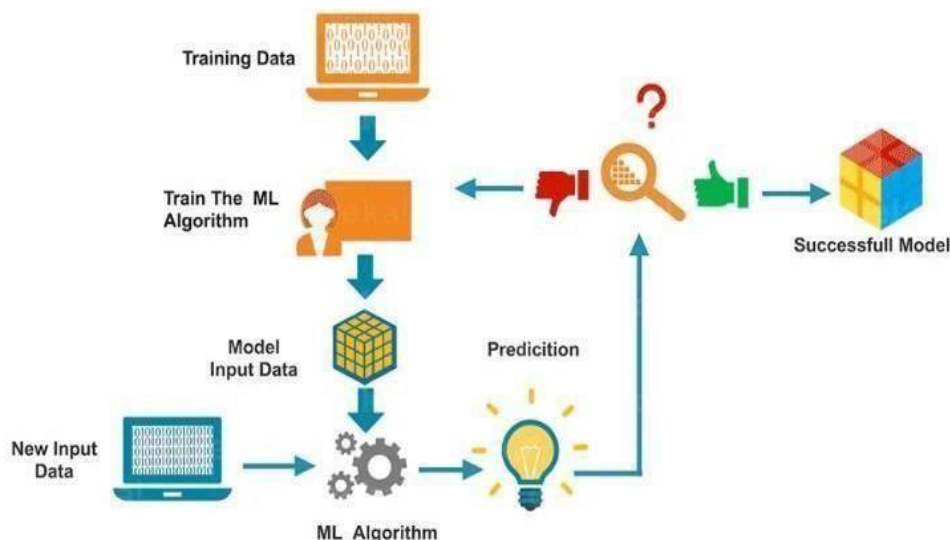
Introduction to Dimensionality Reduction : Principal Component Analysis(PCA), Linear Discriminant Analysis(LDA)

What is Machine Learning?

Machine Learning is a concept which allows the machine to learn from examples and experience, and that too without being explicitly programmed. So instead of you writing the code, what you do is you feed data to the generic algorithm, and the algorithm/ machine builds the logic based on the given data.

How does Machine Learning Work?

Machine Learning algorithm is trained using a training data set to create a model. When new input data is introduced to the ML algorithm, it makes a prediction on the basis of the model. The prediction is evaluated for accuracy and if the accuracy is acceptable, the Machine Learning algorithm is deployed. If the accuracy is not acceptable, the Machine Learning algorithm is trained again and again with an augmented training data set



Types of Machine Learning

Machine learning is sub-categorized to three types:

- **Supervised Learning – Train Me!**
- **Unsupervised Learning – I am self sufficient in learning**
- **Reinforcement Learning – My life My rules! (Hit & Trial)**

Supervised Learning is the one, where you can consider the learning is guided by a teacher. We have a dataset which acts as a teacher and its role is to train the model or the machine. Once the model gets trained it can start making a prediction or decision when new data is given to it.

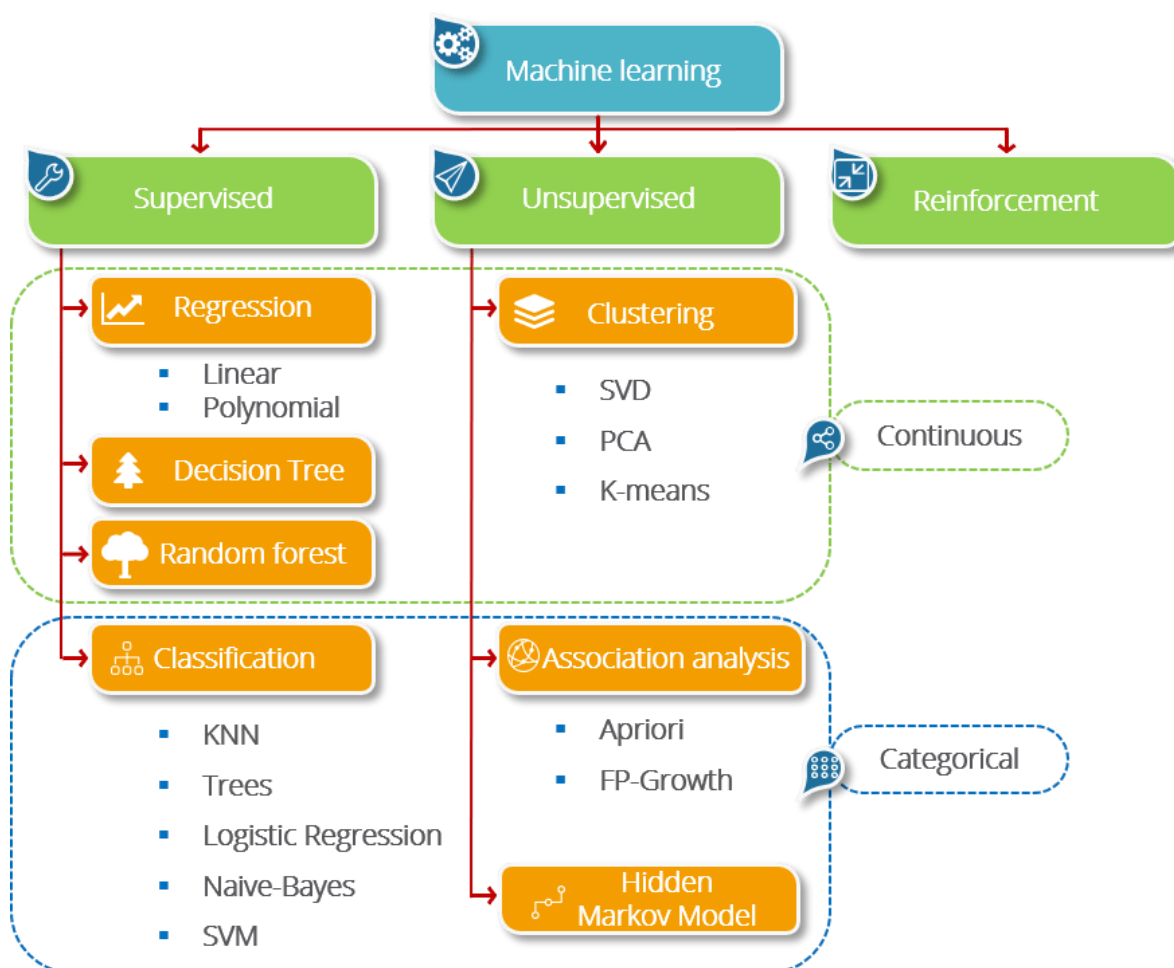
What is Unsupervised Learning?

The model learns through observation and finds structures in the data. Once the model is given a dataset, it automatically finds patterns and relationships in the dataset by creating clusters in it. What it cannot do is add labels to the cluster, like it cannot say this a group of apples or mangoes, but it will separate all the apples from mangoes.

Suppose we presented images of apples, bananas and mangoes to the model, so what it does, based on some patterns and relationships it creates clusters and divides the dataset into those clusters. Now if a new data is fed to the model, it adds it to one of the created clusters.

What is Reinforcement Learning?

It is the ability of an agent to interact with the environment and find out what is the best outcome. It follows the concept of hit and trial method. The agent is rewarded or penalized with a point for a correct or a wrong answer, and on the basis of the positive reward points gained the model trains itself. And again once trained it gets ready to predict the new data presented to it.



Classification of Machine Learning Algorithms Machine Learning algorithms can be classified into:

1. Supervised Algorithms – Linear Regression, Logistic Regression, Support Vector Machine (SVM), DecisionTrees, Random Forest
2. Unsupervised Algorithms – K Means Clustering.
3. Reinforcement Algorithm

1. Supervised Machine Learning Algorithms

In this type of algorithm, the data set on which the machine is trained consists of labelled data or simply said, consists both the input parameters as well as the required output. For example, classifying whether a person is a male or a female. Here male and female will be our labels and our training dataset will already be classified into the

given labels based on certain parameters through which the machine will learn these features and patterns and classify some new input data based on the learning from this training data.

Supervised Learning Algorithms can be broadly divided into two types of algorithms, Classification and Regression. Classification Algorithms

Just as the name suggests, these algorithms are used to classify data into predefined classes or labels. Regression Algorithms

These algorithms are used to determine the mathematical relationship between two or more variables and the level of dependency between variables. These can be used for predicting an output based on the interdependency of two or more variables. For example, an increase in the price of a product will decrease its consumption, which means, in this case, the amount of consumption will depend on the price of the product. Here, the amount of consumption will be called as the dependent variable and price of the product will be called the independent variable. The level of dependency of the amount of consumption on the price of a product will help us predict the future value of the amount of consumption based on the change in prices of the product.

We have two types of regression algorithms: Linear Regression and Logistic Regression

(a) Linear Regression

Linear regression is used with continuously valued variables, like the previous example in which the price of the product and amount of consumption are continuous variables, which means that they can have an infinite number of possible values. Linear regression can also be represented as a graph known as scatter plot, where all the data points of the dependent and independent variables are plotted and a straight line is drawn through them such that the maximum number of points will lie on the line or at a smaller distance from the line. This line – also called the regression line, will then help us determine the relationship between the dependent and independent variables along with which the linear regression equation is formed.

(b) Logistic Regression

The difference between linear and logistic regression is that logistic regression is used with categorical dependent variables (eg: Yes/No, Male/Female, Sunny/Rainy/Cloudy, Red/Blue etc.), unlike the continuous valued variables used in linear regression. Logistic regression helps determine the probability of a certain variable to be in a certain group like whether it is night or day, or whether the colour is red or

blue etc. The graph of logistic regression consists of a non-linear sigmoid function which demonstrates the probabilities of the variables.

2. Unsupervised Machine Learning Algorithms

Unlike supervised learning algorithms, where we deal with labelled data for training, the training data will be unlabelled for Unsupervised Machine Learning Algorithms. The clustering of data into a specific group will be done on the basis of the similarities between the variables. Some of the unsupervised machine learning algorithms are K-means clustering, neural networks.

Another machine learning concept which is extensively used in the field is Neural Networks..

3. Reinforcement Machine Learning Algorithms

Reinforcement Learning is a type of Machine Learning in which the machine is required to determine the ideal behaviour within a specific context, in order to maximize its rewards. It works on the rewards and punishment principle which means that for any decision which a machine takes, it will be either be rewarded or punished due to which it will understand whether or not the decision was correct. This is how the machine will learn to take the correct decisions to maximize the reward in the long run.

For reinforcement algorithm, a machine can be adjusted and programmed to focus more on either the long-term rewards or the short-term rewards. When the machine is in a particular state and has to be the action for the next state in order to achieve the reward, this process is called the Markov Decision Process.

What is Normalization in Machine Learning?

Normalization is a scaling technique in Machine Learning applied during data preparation to change the values of numeric columns in the dataset to use a common scale. It is not necessary for all datasets in a model. It is required only when features of machine learning models have different ranges.

Although **Normalization** is no mandate for all datasets available in machine learning, it is used whenever the attributes of the dataset have different ranges. It helps to enhance the performance and reliability of a machine learning model. In this article, we will discuss in brief various Normalization techniques in machine learning, why it is used, examples of normalization in an ML model, and much more. So, let's start with the definition of Normalization in Machine Learning.

Mathematically, we can calculate normalization with the below formula:

$$X_n = (X - X_{\text{minimum}}) / (X_{\text{maximum}} - X_{\text{minimum}})$$

- X_n = Value of Normalization
- X_{maximum} = Maximum value of a feature
- X_{minimum} = Minimum value of a feature

Example: Let's assume we have a model dataset having maximum and minimum values of feature as mentioned above. To normalize the machine learning model, values are shifted and rescaled so their range can vary between 0 and 1. This technique is also known as **Min-Max scaling**. In this scaling

technique, we will change the feature values as follows:

Case1- If the value of X is minimum, the value of Numerator will be 0; hence Normalization will also be 0.

$$X_n = (X - X_{\text{minimum}}) / (X_{\text{maximum}} - X_{\text{minimum}})$$

Put $X = X_{\text{minimum}}$ in above formula, we get;

$$X_n = (X_{\text{minimum}} - X_{\text{minimum}}) / (X_{\text{maximum}} - X_{\text{minimum}}) = 0$$

Case2- If the value of X is maximum, then the value of the numerator is equal to the denominator; hence Normalization will be 1.

Put $X = X_{\text{maximum}}$ in above formula, we get;

$$X_n = (X_{\text{maximum}} - X_{\text{minimum}}) / (X_{\text{maximum}} - X_{\text{minimum}})$$

What is Dimensionality Reduction?

In machine learning classification problems, there are often too many factors on the basis of which the final classification is done. These factors are basically variables called features. The higher the number of features, the harder it gets to visualize the training set and then work on it. Sometimes, most of these features are correlated, and hence redundant. This is where dimensionality reduction algorithms come into play. Dimensionality reduction is the process of reducing the number of random variables under consideration, by obtaining a set of principal variables. It can be divided into feature selection and feature extraction.

Why is Dimensionality Reduction important in Machine Learning and Predictive Modeling?

An intuitive example of dimensionality reduction can be discussed through a simple e-mail classification problem, where we need to classify whether the e-mail is spam or not. This can involve a large number of features, such as whether or not the e-mail has a generic title, the content of the e-mail, whether the e-mail uses a template, etc. However, some of these features may overlap. In another condition, a classification problem that relies on both humidity and rainfall can be collapsed into just one underlying feature, since both of the aforementioned are correlated to a high degree. Hence, we can reduce the number of features in such problems. A 3-D classification problem can be hard to visualize, whereas a 2-D one can be mapped to a simple 2 dimensional space, and a 1-D problem to a simple line. The below figure illustrates this concept, where a 3-D feature space is split into two 1-D feature spaces, and later, if found to be correlated, the number of features can be reduced even further

Components of Dimensionality Reduction

There are two components of dimensionality reduction:

- **Feature selection:** In this, we try to find a subset of the original set of variables, or features, to get a smaller subset which can be used to model the problem. It usually involves three ways:
 1. Filter
 2. Wrapper
 3. Embedded
- **Feature extraction:** This reduces the data in a high dimensional space to a lower dimension space, i.e. a space with lesser no. of dimensions.

Methods of Dimensionality Reduction

The various methods used for dimensionality reduction include:

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Generalized Discriminant Analysis (GDA)

Dimensionality reduction may be both linear or non-linear, depending upon the method used. The prime linear method, called Principal Component Analysis, or PCA, is discussed

Principal Component Analysis

This method was introduced by Karl Pearson. It works on a condition that while the data in a higher dimensional space is mapped to data in a lower dimension space, the variance of the data in the lower dimensional space should be maximum.

It involves the following steps:

- Construct the covariance matrix of the data.
- Compute the eigenvectors of this matrix.
- Eigenvectors corresponding to the largest eigenvalues are used to reconstruct a large fraction of variance of the original data.

Hence, we are left with a lesser number of eigenvectors, and there might have been some data loss in the process. But, the most important variances should be retained by the remaining eigenvectors.

Advantages of Dimensionality Reduction

- It helps in data compression, and hence reduced storage space.
- It reduces computation time.
- It also helps remove redundant features, if any.

Disadvantages of Dimensionality Reduction

- It may lead to some amount of data loss.
- PCA tends to find linear correlations between variables, which is sometimes undesirable.
- PCA fails in cases where mean and covariance are not enough to define datasets.
- We may not know how many principal components to keep- in practice, some thumb rules are applied.
- Explain fold
- based filtering

3. Kernel Principal Component Analysis

There are a lot of machine learning problems which are nonlinear, and the use of nonlinear feature mappings can help to produce new features which make prediction problems linear. In this section we will discuss the following idea: transformation of the dataset to a new higher-dimensional (in some cases infinite-dimensional) feature space and the use of PCA in that space in order to produce uncorrelated features. Such a method is called **Kernel Principal Component Analysis** or **KPCA**.

Let us denote a covariance matrix in a new feature space as

$$\varphi: \mathbb{R}^D \rightarrow \mathbb{F},$$

$$\mathbf{C} = \frac{1}{N} \Phi^T \Phi,$$

where $\Phi = \{\phi(\mathbf{x}_n)\}_{n=1}^N$. Will consider that the dimensionality of the feature space equals to . Eigendecomposition of is given by

$$\mathbf{C}\mathbf{v}_i = \lambda_i \mathbf{v}_i, \forall i = 1, \dots, M.$$

By the definition of

And therefore $\forall i = 1, \dots, M$

$$\frac{1}{N} \sum_{n=1}^N \phi(\mathbf{x}_n) \{\phi(\mathbf{x}_n)^\top \mathbf{v}_i\} = \lambda_i \mathbf{v}_i.$$

It is obviously to see, that is a linear combination of $\phi(\mathbf{x}_n)$ and thus can be written as

$$\mathbf{v}_i = \sum_{n=1}^N a_{i_n} \phi(\mathbf{x}_n).$$

Substituting it to the equation above and writing it in a matrix notation, we get

$$\mathbf{K}\mathbf{a}_i = \lambda_i N \mathbf{a}_i,$$

where is a Gram matrix in , $\mathbf{K}_{ij} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ and are column-vectors with elements Eigenvectors of should be orthonormal, therefore, we get the following:

$$N \lambda_i \mathbf{a}_i^\top \mathbf{a}_i = 1.$$

Having eigenvectors of , we can get the projection of an item $\mathbf{x} \in \mathbb{R}^D$ on -th eigenvector:

$$z_i(\mathbf{x}) = \sum_{n=1}^N a_{i_n} \phi(\mathbf{x})^\top \phi(\mathbf{x}_n).$$

So far, we have assumed that the mapping $\varphi(\cdot)$ is known. From the equations above, we can see, that only a thing that we need for the data transformation is the eigendecomposition of a Gram matrix . Dot products, which are its elements can be defined without any definition of $\varphi(\cdot)$. The function defining such dot products in some Hilbert space is called kernel. Kernels are satisfied by the Mercer's theorem. There are many different types of kernels, there are several popular:

1. Linear: ;
2. Gaussian: ;
3. Polynomial: .

Using a kernel function we can write new equation for a projection of some data item onto -th eigenvector:

$$z_i(\mathbf{x}) = \sum_{n=1}^N a_{i_n} k(\mathbf{x}, \mathbf{x}_n)$$

So far, we have assumed that the columns of $\mathbf{K}$ have zero mean. Using

,

and substituting it to the equation for $\mathbf{K}_c$, we get

$$\mathbf{K}_c = \mathbf{K} - \mathbf{1}_N \mathbf{K} - \mathbf{K} \mathbf{1}_N + \mathbf{1}_N \mathbf{K} \mathbf{1}_N,$$

where $\mathbf{1}_N$ is a matrix $\mathbf{1}_N$, where each element equals to $\frac{1}{N}$.

Summary: Now we are ready to write the whole sequence of steps to perform KPCA:

1. Calculate $\mathbf{K}$.
2. Calculate $\mathbf{K}_c$.
3. Find the eigenvectors $\mathbf{a}_i$ of $\mathbf{K}_c$ corresponding to nonzero eigenvalues λ_i and normalize them:
 $\mathbf{a}_i = \frac{1}{\sqrt{\lambda_i N}} \mathbf{a}_i$.
4. Sort found eigenvectors in the descending order of corresponding eigenvalues.
5. Perform projections onto the given subset of eigenvectors.

The method described above requires to define the number of components, the kernel and its parameters. It should be noted, that the number of nonlinear principal components in the general case is infinite, but since we are computing the eigenvectors of a matrix, at maximum we can calculate nonlinear principal components.

UNIT-II

Supervised Learning – I (Regression/Classification)

Regression models: Simple Linear Regression, multiple linear Regression. Cost Function, Gradient Descent, Performance Metrics: Mean Absolute Error(MAE), Mean Squared Error(MSE) R-Squared error, Adjusted R Square.

Classification models: Decision Trees-ID3, CART, Naive Bayes, K-Nearest-Neighbours (KNN), Logistic Regression, Multinomial Logistic Regression Support Vector Machines (SVM) - Nonlinearity and Kernel Methods

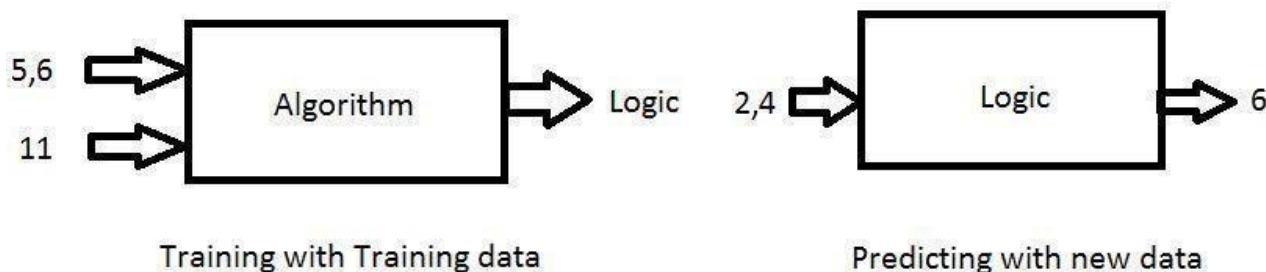
Supervised and unsupervised are mostly used by a lot machine learning engineers and data geeks. Reinforcement learning is really powerful and complex to apply for problems.

Supervised learning

as we know machine learning takes data as input. lets call this data **Training data**

The training data includes both *Inputs* and *Labels(Targets)*

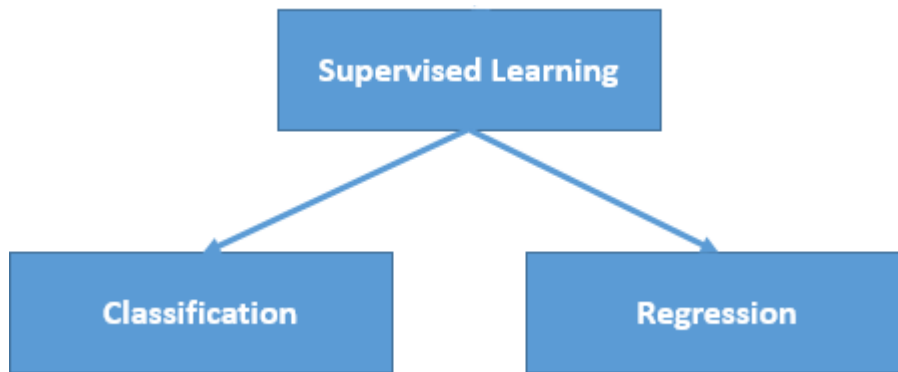
what are Inputs and Labels(Targets)?? for example addition of two numbers $a=5, b=6$ result $=11$, *Inputs* are 5,6 and *Target* is 11



We first train the model with the lots of training data(inputs&targets)then with new data and the logic we got before we predict the output

(Note :We don't get exact 6 as answer we may get value which is close to 6 based on training data and algorithm)

This process is called *Supervised Learning* which is really fast and accurate.



Types of Supervised learning

Regression: This is a type of problem where we need to predict the *continuous-response* value (ex : above we predict number which can vary from -infinity to +infinity)

Some examples are

- what is the price of house in a specific city?
- what is the value of the stock?
- how many total runs can be on board in a cricket game?etc... there are tons of things we can predict if we wish.

Classification: This is a type of problem where we predict the *categorical response* value where the data can be separated into specific “**classes**” (ex: we predict one of the values in a set of values).

Some examples are :

- this mail is spam or not?
- will it rain today or not?
- is this picture a cat or not?

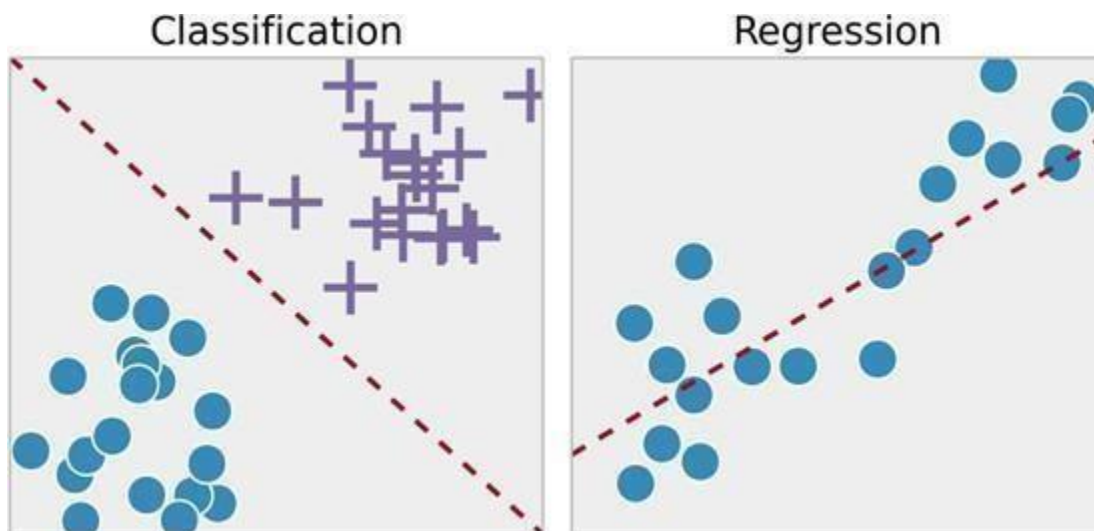
Basically ‘Yes/No’ type questions called **binary classification**.

Other examples are :

- this mail is spam or important or promotion?
- is this picture a cat or a dog or a tiger?

This type is called **multi-class classification**.

Here is the final picture



Classification separates the data, Regression fits the data

That's all for supervised learning.

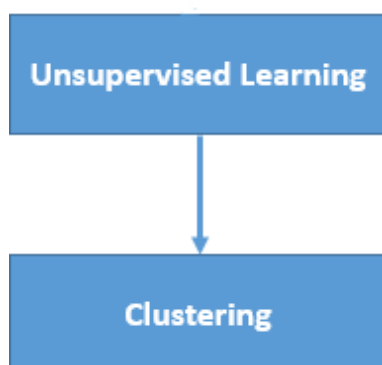
Unsupervised learning

The training data does not include Targets here so we don't tell the system where to go, the system has to understand itself from the data we give.

Here training data is not structured (contains noisy data, unknown data and etc..)

Unsupervised process

There are also different types for unsupervised learning like Clustering and anomaly detection (clustering is pretty famous)

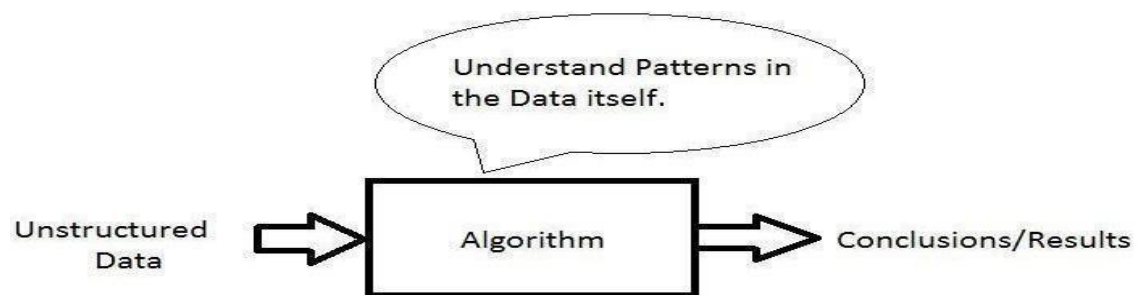


Clustering: This is a type of problem where we group similar things together.

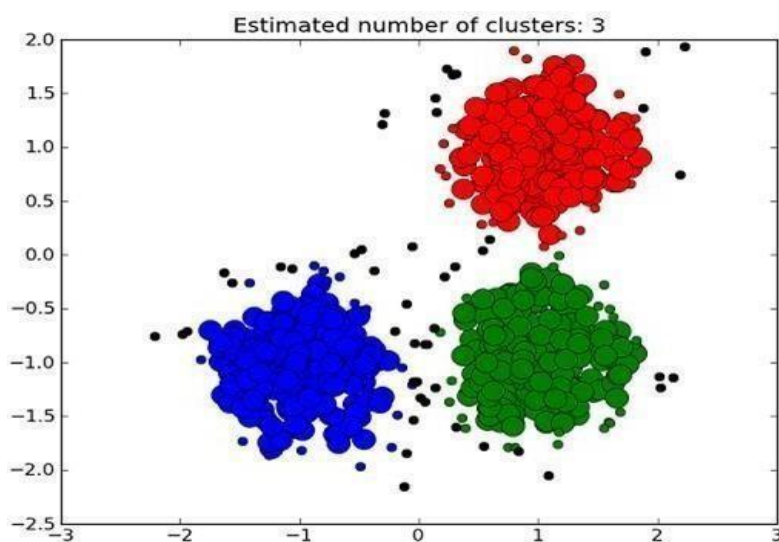
Bit similar to multi class classification but here we don't provide the labels, the system understands from data itself and cluster the data.

Some examples are :

given news articles, cluster into different types of news



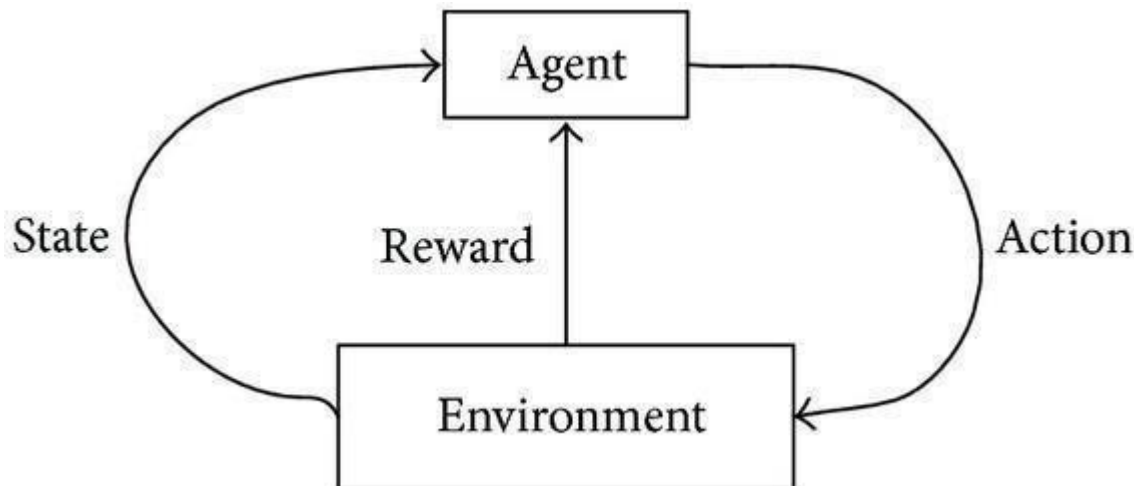
- given a set of tweets ,cluster based on content of tweet
- given a set of images, cluster them into different objects



Clustering with 3 clusters

Unsupervised learning is bit difficult to implement and its not used as widely as supervised.

Reinforcement Learning is a type of *Machine Learning*, and thereby also a branch of *Artificial Intelligence*. It allows machines and software agents to automatically determine the ideal behavior within a specific context, in order to maximize its performance. Simple reward feedback is required for the agent to learn its behavior; this is known as thereinforcement signal.



There are many different algorithms that tackle this issue. As a matter of fact, Reinforcement Learning is defined by a specific type of problem, and all its solutions are classed as Reinforcement Learning algorithms. In the problem, an agent is supposed to decide the best action to select based on his current state. When this step is repeated, the problem is known as a *Markov Decision Process*.

In order to produce intelligent programs (also called agents), reinforcement learning goes through the following steps:

1. Input state is observed by the agent.
2. Decision making function is used to make the agent perform an action.
3. After the action is performed, the agent receives reward or reinforcement from the environment.
4. The state-action pair information about the reward is stored.

List of Common Algorithms

- Q-Learning
 - Temporal Difference (TD)
- Deep Adversarial Networks

Use cases:

Some applications of the *reinforcement learning algorithms* are computer played board games (Chess, Go), robotic hands, and self-driving cars.



Regression analysis is a statistical method to model the relationship between a dependent (target) and independent (predictor) variables with one or more independent variables. More specifically, Regression analysis helps us to understand how the value of the dependent variable is changing corresponding to an independent variable when other independent variables are held fixed. It predicts continuous/real values such as **temperature, age, salary, price**, etc.

We can understand the concept of regression analysis using the below example:

Example: Suppose there is a marketing company A, who does various advertisement every year and get sales on that. The below list shows the advertisement made by the company in the last 5 years and the corresponding sales:

Advertisement	Sales
\$90	\$1000
\$120	\$1300
\$150	\$1800
\$100	\$1200
\$130	\$1380
\$200	??

Now, the company wants to do the advertisement of \$200 in the year 2019 **and wants to know the prediction about the sales for this year**. So to solve such type of prediction problems in machine learning, we need regression analysis.

Regression is a [supervised learning technique](#) which helps in finding the correlation between variables and enables us to predict the continuous output variable based on the one or more predictor variables. It is mainly used for **prediction, forecasting, time series modeling, and determining the causal-effect relationship between variables**.

In Regression, we plot a graph between the variables which best fits the given datapoints, using this plot, the machine learning model can make predictions about the data. In simple words, ***"Regression shows a line or curve that passes through all the datapoints on target-predictor graph in such a way that the vertical distance between the datapoints and the regression line is minimum."*** The distance between datapoints and line tells whether a model has captured a strong relationship or not.

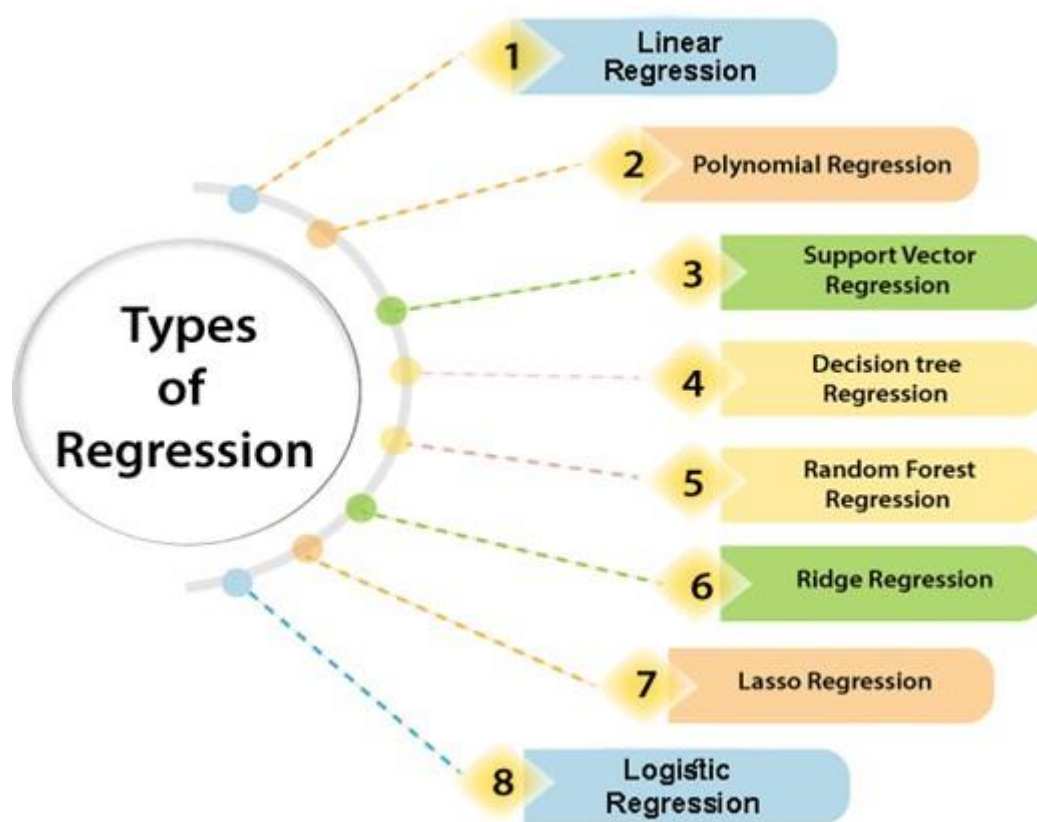
Some examples of regression can be as:

- Prediction of rain using temperature and other factors
 - Determining Market trends
- Prediction of road accidents due to rash driving.

Types of Regression

There are various types of regressions which are used in data science and machine learning. Each type has its own importance on different scenarios, but at the core, all the regression methods analyze the effect of the independent variable on dependent variables. Here we are discussing some important types of regression which are given below:

- **Linear Regression**
- **Logistic Regression**
- **Polynomial Regression**
- **Support Vector Regression**
- **Decision Tree Regression**
- **Random Forest Regression**
- **Ridge Regression**
- **Lasso Regression:**



COST FUNCTION

The cost function also called the **loss function**, computes the difference or distance between actual output and predicted output.

It determines the performance of a Machine Learning Model using a single real number, known as **cost value/model error**. This value depicts the average error between the actual and predicted outputs.

On a broader level, the **cost function evaluates how accurately the model maps the input and output data relationship**. Understanding the consistencies and inconsistencies in the model's performance for a given dataset is critical. *These models work with real-world applications and the slightest error can impact the overall projection and incur losses.*

Types of Cost Function

Depending upon the given dataset, use case, problem, and purpose, there are primarily three types of cost functions as follows:

Regression Cost Function

In simpler words, Regression in Machine Learning is the method of retrograding from ambiguous & hard-to-interpret data to a more explicit & meaningful model.

It is a predictive modeling technique to examine the relationship between independent features and dependent outcomes.

The Regression models operate on serial data or variables. Therefore, they predict continuous outcomes like weather forecasts, probability of loan approvals, car & home costs, the expected employees' salary, etc.

When the cost function deals with the problem statement of the Regression Model, it is known as Regression Cost Function. It computes the error as the distance between the actual output and the predicted output.

The Regression Cost Functions are the simplest and fine-tuned for linear progression. The most common among them are:

i. Mean Error (ME)

ME is the most straightforward approach and acts as a foundation for other Regression Cost Functions. It computes the error for every training dataset and calculates the mean of all derived errors.

ME is usually not suggested because the error values are either positive or negative. During mean calculation, they cancel each other and give a zero-mean error outcome.

ii. Mean Absolute Error (MAE)

MAE, also known as L1 Loss, overcomes the drawback of Means Error (ME) mentioned above. It computes the absolute distance between the actual output and predicted output and is insensitive to anomalies. In addition, MAE does not penalize high errors caused by these anomalies.

Overall, it effortlessly operates the dataset with any anomalies and predicts outcomes with better precision.

However, MAE comes with the drawback of being non-differentiable at zero. Thus, fail to perform well in Loss Function Optimization Algorithms that involve differentiation to evaluate optimal coefficients.

iii. Mean Squared Error (MSE)

MSE, also known as L2 Loss, is used most frequently and successfully improves the drawbacks of both ME and MAE. It computes the "square" of the distance between the actual output and predicted output, preventing negative error possibilities.

Due to squaring errors, MSE penalizes high errors caused by the anomalies and is beneficial to Loss Function Optimization Algorithms for evaluating optimal coefficients.

Its more enhanced extensions are Root Mean Squared Error (RMSE) and Root Mean Squared Logarithmic Error (RMSLE).

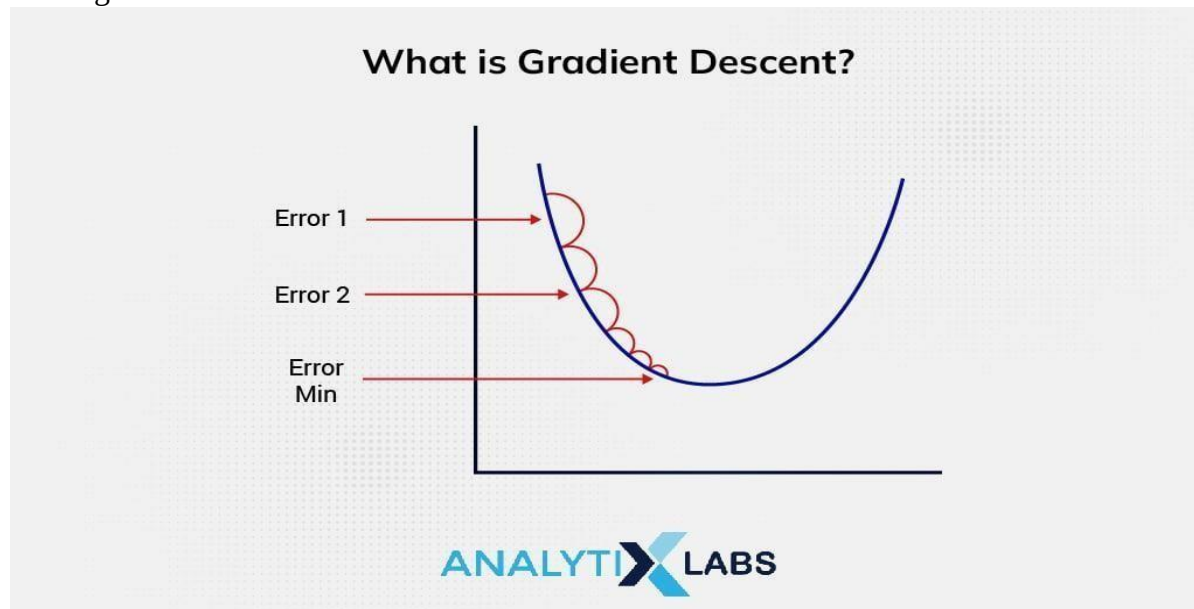
Unlike MAE, MSE is extensively sensitive to anomalies wherein squaring errors quantify it multiple times (into a larger error).

What is Gradient Descent?

In machine learning models, training periods are one of the critical phases to make the model more accurate. To understand how precise a model works, you can just run it across required case

scenarios. But to know how wrong the model is, or what are the points that cause more faults in the output, a comparative function is required.

A cost function is a single real number used to indicate the distance between actual output and predicted output in an ML model. To improve the whole model, when this cost function is optimized through an algorithm to find the minimum possible number of errors in the model, it is called gradient descent.



Gradient Descent is the productive optimization algorithm that minimizes the cost function and generates the most promising results. The reason is its ability to identify the slightest potential error in the model.

It is possible to have different cost values at distinct positions in a model. Thus, for sustainable utilization of resources (without wastage), immediate steps need to be taken to minimize model errors. Here, Gradient Descent iteratively tweaks the model with optimal coefficients (parameters) that help to downsize the cost function.

Decision Trees

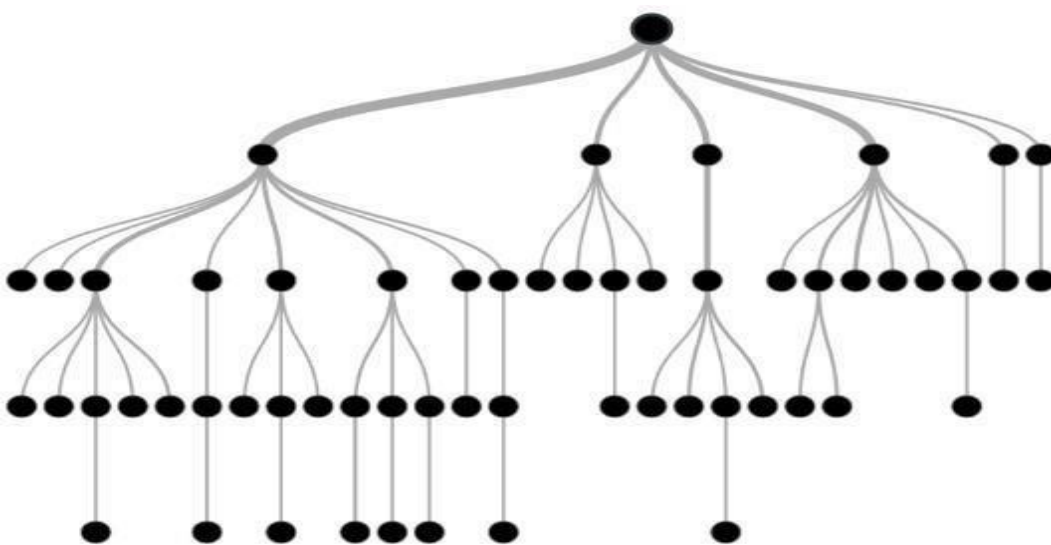
What is a Decision Tree?

A decision tree is a **non-parametric supervised learning algorithm for classification and regression tasks**. It has a hierarchical tree structure consisting of a root node, branches, internal nodes, and leaf nodes. Decision trees are used for classification and regression tasks, providing easy-to-understand models.

A decision tree is a hierarchical model used in decision support that depicts decisions and their potential outcomes, incorporating chance events, resource expenses, and utility. This algorithmic

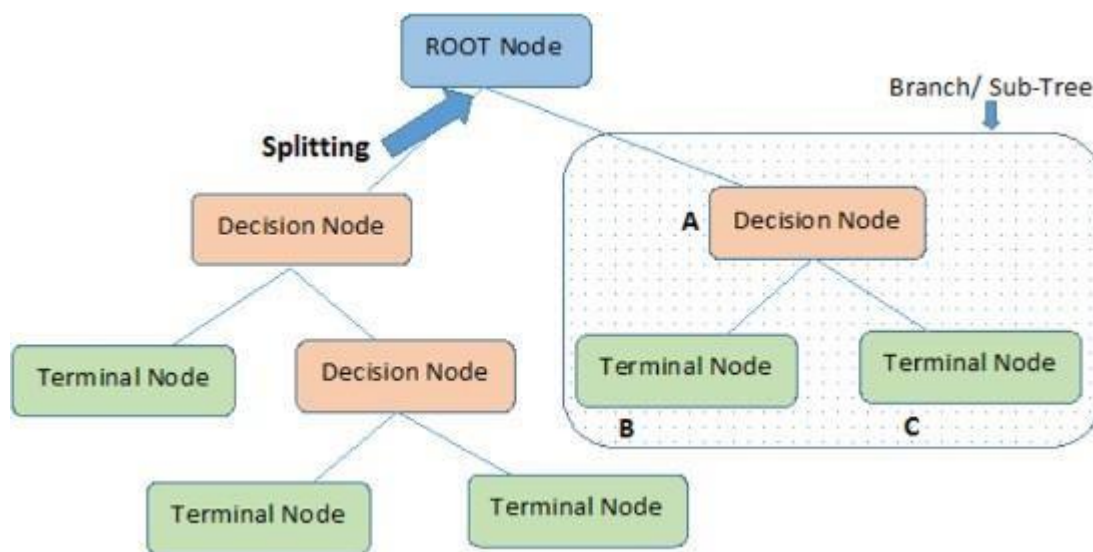
model utilizes conditional control statements and is non-parametric, supervised learning, useful for both classification and regression tasks. The tree structure is comprised of a root node, branches, internal nodes, and leaf nodes, forming a hierarchical, tree-like structure.

It is a tool that has applications spanning several different areas. Decision trees can be used for classification as well as regression problems. The name itself suggests that it uses a flowchart like a tree structure to show the predictions that result from a series of feature-based splits. It starts with a root node and ends with a decision made by leaves.



- **Root Node:** The initial node at the beginning of a decision tree, where the entire population or dataset starts dividing based on various features or conditions.
- **Decision Nodes:** Nodes resulting from the splitting of root nodes are known as decision nodes. These nodes represent intermediate decisions or conditions within the tree.
- **Leaf Nodes:** Nodes where further splitting is not possible, often indicating the final classification or outcome. Leaf nodes are also referred to as terminal nodes.
- **Sub-Tree:** Similar to a subsection of a graph being called a sub-graph, a sub-section of a decision tree is referred to as a sub-tree. It represents a specific portion of the decision tree.
- **Pruning:** The process of removing or cutting down specific nodes in a decision tree to prevent overfitting and simplify the model.

- **Branch / Sub-Tree:** A subsection of the entire decision tree is referred to as a branch or sub-tree. It represents a specific path of decisions and outcomes within the tree.
- **Parent and Child Node:** In a decision tree, a node that is divided into sub-nodes is known as a parent node, and the sub-nodes emerging from it are referred to as child nodes. The parent node represents a decision or condition, while the child nodes represent the potential outcomes or further decisions based on that condition.



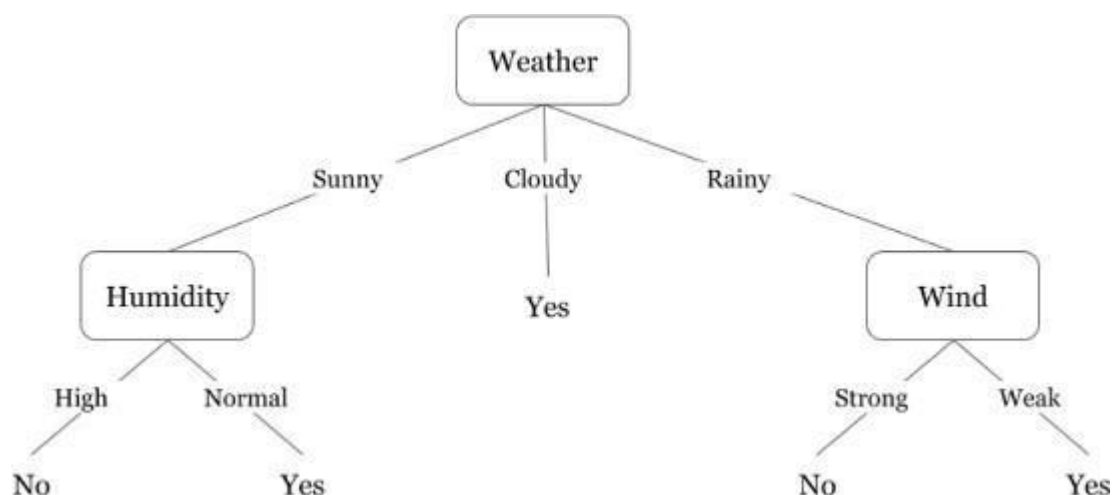
Example of Decision Tree

Let's understand decision trees with the help of an example:

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No

Decision trees are upside down which means the root is at the top and then this root is split into various several nodes. Decision trees are nothing but a bunch of if-else statements in layman terms. It checks if the condition is true and if it is then it goes to the next node attached to that decision.

In the below diagram the tree will first ask what is the weather? Is it sunny, cloudy, or rainy? If yes then it will go to the next feature which is humidity and wind. It will again check if there is a strong wind or weak, if it's a weak wind and it's rainy then the person may go and play.



Did you notice anything in the above flowchart? We see that if the *weather is cloudy* then we must go to play. Why didn't it split more? Why did it stop there?

To answer this question, we need to know about few more concepts like entropy, information gain, and Gini index. But in simple terms, I can say here that the output for the training dataset is always yes for cloudy weather, since there is no disorderliness here we don't need to split the node further.

The goal of machine learning is to decrease uncertainty or disorders from the dataset and for this, we use decision trees.

Now you must be thinking how do I know what should be the root node? what should be the decision node? when should I stop splitting? To decide this, there is a metric called "Entropy" which is the amount of uncertainty in the dataset.

How decision tree algorithms work?

Decision Tree algorithm works in simpler steps

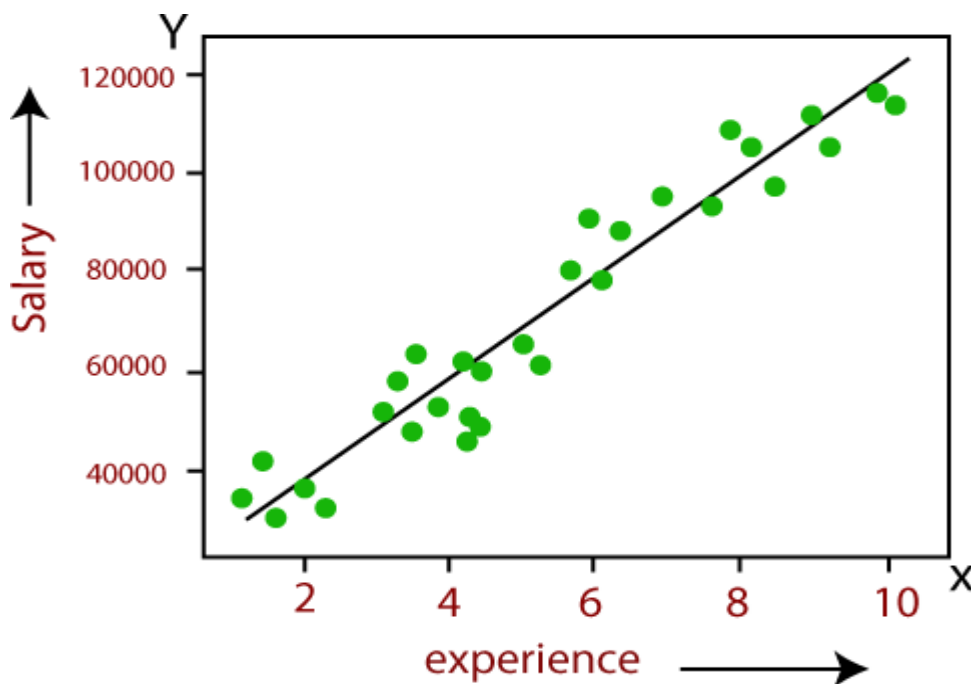
1. **Starting at the Root:** The algorithm begins at the top, called the "root node," representing the entire dataset.
2. **Asking the Best Questions:** It looks for the most important feature or question that splits the data into the most distinct groups. This is like asking a question at a fork in the tree.

3. **Branching Out:** Based on the answer to that question, it divides the data into smaller subsets, creating new branches. Each branch represents a possible route through the tree.
4. **Repeating the Process:** The algorithm continues asking questions and splitting the data at each branch until it reaches the final “leaf nodes,” representing the predicted outcomes or classifications.

Linear Regression:

- Linear regression is a statistical regression method which is used for predictive analysis.
- It is one of the very simple and easy algorithms which works on regression and shows the relationship between the continuous variables.
- It is used for solving the regression problem in machine learning.

- Linear regression shows the linear relationship between the independent variable (X-axis) and the dependent variable (Y-axis), hence called linear regression.
- If there is only one input variable (x), then such linear regression is called **simple linear regression**. And if there is more than one input variable, then such linear regression is called **multiple linear regression**.
- The relationship between variables in the linear regression model can be explained using the below image. Here we are predicting the salary of an employee on the basis of **the year of experience**.



Below is the mathematical equation for Linear regression: $Y = aX + b$

Here,

Y = dependent variables (target

variables), X =

Independent

Some popular applications of linear regression are:

- **Analyzing trends and sales estimates**
- **Salary forecasting**
- **Real estate prediction**
- **Arriving at ETAs in traffic.**
-

K Nearest Neighbors– Classification

K nearest neighbors is a simple algorithm that stores all available cases and classifies new cases based on a similarity measure (e.g., distance functions). KNN has been used in statistical estimation and pattern recognition already in the beginning of 1970's as a non-parametric technique

ALGORITHM

A case is classified by a majority vote of its neighbors, with the case being assigned to the class most common amongst its K nearest neighbors measured by a distance function. If $K = 1$, then the case is simply assigned to the class of its nearest neighbor

Distance functions

Euclidean	$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$
Manhattan	$\sum_{i=1}^k x_i - y_i $
Minkowski	$\left(\sum_{i=1}^k (x_i - y_i)^q \right)^{1/q}$

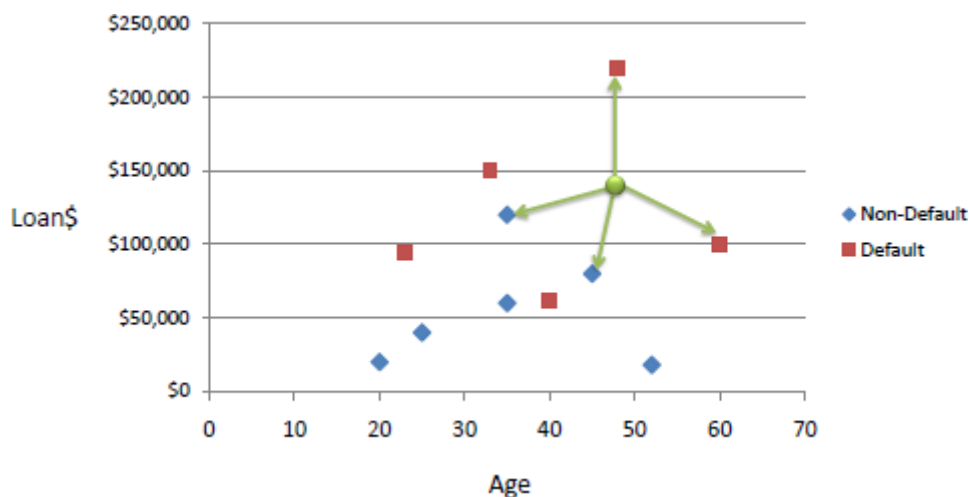
Hamming Distance

$$D_H = \sum_{i=1}^k |x_i - y_i|$$

$$x = y \Rightarrow D = 0$$

$$x \neq y \Rightarrow D = 1$$

X	Y	Distance
Male	Male	0
Male	Female	1



Age	Loan	Default	Distance
25	\$40,000	N	102000
35	\$60,000	N	82000
45	\$80,000	N	62000
20	\$20,000	N	122000
35	\$120,000	N	22000
52	\$18,000	N	124000
23	\$95,000	Y	47000
40	\$62,000	Y	80000
60	\$100,000	Y	42000
48	\$220,000	Y	78000
33	\$150,000	Y	8000
48	\$142,000	?	

Euclidean Distance

$$D = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

what is a classifier?

A classifier is a machine learning model that is used to discriminate different objects based on certain features.

Principle of Naive Bayes Classifier:

A Naive Bayes classifier is a probabilistic machine learning model that's used for classification task. The crux of the classifier is based on the Bayes theorem.

Bayes Theorem:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Using Bayes theorem, we can find the probability of **A** happening, given that **B** has occurred. Here, **B** is the evidence and **A** is the hypothesis. The assumption made here is that the predictors/features are independent. That is presence of one particular feature does not affect the other. Hence it is called naive.

Example:

Let us take an example to get some better intuition. Consider the problem of playing golf. The dataset is represented as below.

	OUTLOOK	TEMPERATURE	HUMIDITY	WINDY	PLAY GOLF
0	Rainy	Hot	High	False	No
1	Rainy	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Sunny	Mild	High	False	Yes
4	Sunny	Cool	Normal	False	Yes
5	Sunny	Cool	Normal	True	No
6	Overcast	Cool	Normal	True	Yes
7	Rainy	Mild	High	False	No
8	Rainy	Cool	Normal	False	Yes
9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Overcast	Hot	Normal	False	Yes
13	Sunny	Mild	High	True	No

We classify whether the day is suitable for playing golf, given the features of the day. The columns represent these features and the rows represent individual entries. If we take the first row of the dataset, we can observe

that is not suitable for playing golf if the outlook is rainy, temperature is hot, humidity is high and it is not windy. We make two assumptions here, one as stated above we consider that these predictors are independent. That is, if the temperature is hot, it does not necessarily mean that the humidity is high. Another assumption made here is that all the predictors have an equal effect on the outcome. That is, the day being windy does not have more importance in deciding to play golf or not.

According to this example, Bayes theorem can be rewritten as:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

The variable y is the class variable(play golf), which represents if it is suitable to play golf or not given the conditions. Variable X represent the parameters/features.

X is given as,

$$X = (x_1, x_2, x_3, \dots, x_n)$$

Here $x_1, x_2, \dots, x_n$ represent the features, i.e they can be mapped to outlook, temperature, humidity and windy. By substituting for X and expanding using the chain rule we get,

$$P(y|x_1, \dots, x_n) = \frac{P(x_1|y)P(x_2|y)\dots P(x_n|y)P(y)}{P(x_1)P(x_2)\dots P(x_n)}$$

Now, you can obtain the values for each by looking at the dataset and substitute them into the equation. For all entries in the dataset, the denominator does not change, it remain static. Therefore, the denominator can be removed and a proportionality can be introduced.

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

In our case, the class variable(y) has only two outcomes, yes or no. There could be cases where the classification could be multivariate. Therefore, we need to find the class y with maximum probability.

$$y = \operatorname{argmax}_y P(y) \prod_{i=1}^n P(x_i|y)$$

Using the above function, we can obtain the class, given the predictors. Types of Naive Bayes Classifier:

Multinomial Naive Bayes:

This is mostly used for document classification problem, i.e whether a document belongs to the category of sports, politics, technology etc. The features/predictors used by the classifier are the frequency of the words present in the document.

Bernoulli Naive Bayes:

This is similar to the multinomial naive bayes but the predictors are boolean variables. The parameters that we use to predict the class variable take up only values yes or no, for example if a word occurs in the text or not.

Gaussian Naive Bayes:

When the predictors take up a continuous value and are not discrete, we assume that these values are sampled from a gaussian distribution.

Conclusion:

Naive Bayes algorithms are mostly used in sentiment analysis, spam filtering, recommendation systems etc. They are fast and easy to implement but their biggest disadvantage is that the requirement of predictors to be independent. In most of the real life cases, the predictors are dependent, this hinders the performance of the classifier.

Naive Bayes Classifier technique is based on the so-called Bayesian theorem and is particularly suited when the dimensionality of the inputs is high. Despite its simplicity, Naive Bayes can often outperform more sophisticated classification methods.



To demonstrate the concept of Naïve Bayes Classification, consider the example displayed in the illustration above. As indicated, the objects can be classified as either GREEN or RED. Our task is to classify new cases as they arrive, i.e., decide to which class label they belong, based on the currently existing objects.

Since there are twice as many GREEN objects as RED, it is reasonable to believe that a new case (which hasn't been observed yet) is twice as likely to have membership GREEN rather than RED. In the Bayesian analysis, this belief is known as the prior probability. Prior probabilities are based on previous experience, in this case the percentage of GREEN and RED objects, and often used to predict outcomes before they actually happen.

Thus, we can write:

Although the prior probabilities indicate that X may belong to GREEN (given that there are twice as many GREEN compared to RED) the likelihood indicates otherwise; that the class membership of X is RED (given that there are more RED objects in the vicinity of X than GREEN). In the Bayesian analysis, the final classification is produced by combining both sources of information, i.e., the prior and the likelihood, to form a posterior probability using the so-called Bayes' rule (named after Rev. Thomas Bayes 1702-1761).

Posterior probability of X being GREEN $\propto$

Prior probability of GREEN $\times$ Likelihood of X given GREEN

$$= \frac{4}{6} \times \frac{1}{40} = \frac{1}{60}$$

Posterior probability of X being RED $\propto$

Prior probability of RED $\times$ Likelihood of X given RED

$$= \frac{2}{6} \times \frac{3}{20} = \frac{1}{20}$$

Assume that you are given a characteristic information of 10,000 people living in your town. You are asked to study them and come up with the algorithm which should be able to tell whether a new person coming to the town is male or a female.

Primarily you are given information about:

Skin colour Hair length Weight Height

Based on the information you can divide the information in such a way that it somehow indicates the characteristics of Males vs. Females.

Below is a hypothetical tree designed out of this data:



The tree shown above divides the data in such a way that we gain the maximum information, to understand the tree

– If a person's hair length is less than 5 Inches, weight greater than 55 KGs then there are 80% chances for that person being a Male.

If you are familiar with Predictive Modelling e.g., Logistic Regression, Random Forest etc. – You might be wondering what is the difference between a Logistic Model and Decision Tree! Because in both the algorithms we are trying to predict a categorical variable.

There are a few fundamental differences between both but ideally both the approaches should give you the same results. The best use of Decision Trees is when your solution requires a representation. For example, you are working for a Telecom Operator and building a solution using which a call center agent can take a

decision whether to pitch for an upsell or not!

There are very less chances that a call center executive will understand the Logistic Regression or the equations, but using a more visually appealing solution you might gain a better adoption from your call center team.

How does Decision Tree work?

There are multiple algorithms written to build a decision tree, which can be used according to the problem characteristics you are trying to solve. Few of the commonly used algorithms are listed below:

ID3 C4.5 CART

CHAID (CHi-squared Automatic Interaction Detector)MARS

Conditional Inference Trees

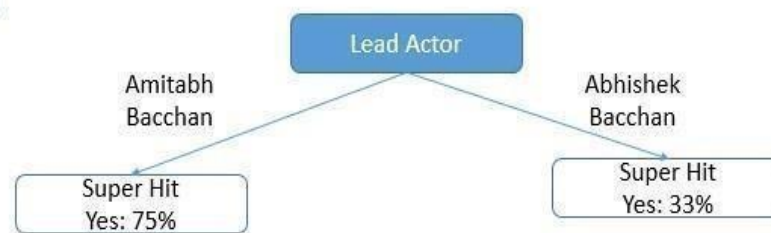
Though the methods are different for different decision tree building algorithms but all of them works on the principle of Greediness. Algorithms try to search for a variable which give the maximum information gain or divides the data in the most homogenous way.

For an example, consider the following hypothetical dataset which contains Lead Actor and Genre of a movie alongwith the success on box office:

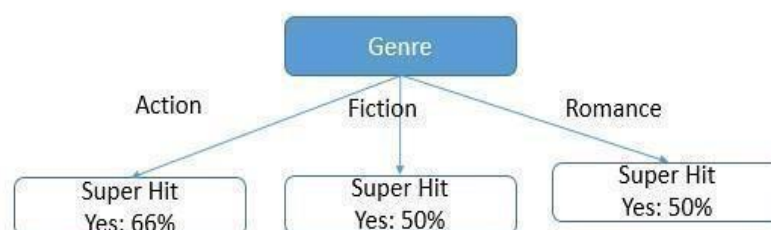
Lead Actor	Genre	Hit(Y/N)
Amitabh Bacchan	Action	Yes
Amitabh Bacchan	Fiction	Yes
Amitabh Bacchan	Romance	No
Amitabh Bacchan	Action	Yes
Abhishek Bacchan	Action	No
Abhishek Bacchan	Fiction	No
Abhishek Bacchan	Romance	Yes

Let say, you want to identify the success of the movie but you can use only one variable – There are the followingtwo ways in which this can be done:

Method 1



Method 2



You can clearly observe that Method 1 (Based on lead actor) splits the data best while the second method (Based on Genre) have produced mixed results. Decision Tree algorithms do similar things when it comes to select variables.

There are various metrics which decision trees use in order to find out the best split variables. We'll go through them one by one and try to understand, what do they mean?

Entropy & Information Gain

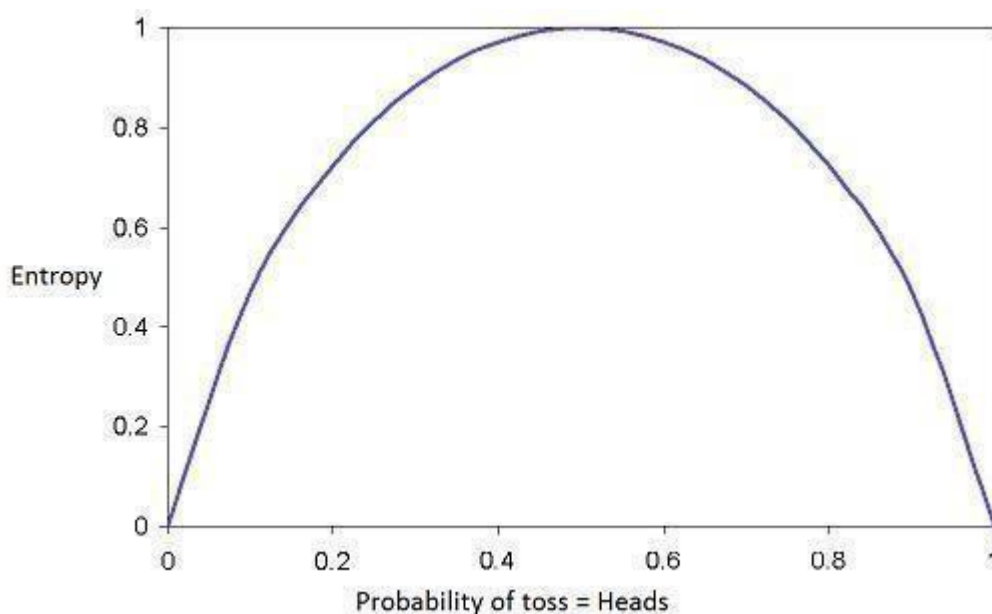
The word Entropy is borrowed from Thermodynamics which is a measure of variability or chaos or randomness. Shannon extended the thermodynamic entropy concept in 1948 and introduced it into statistical studies and suggested the following formula for statistical entropy:

$$H = - \sum_{j=1}^n P_j \ln P_j$$

Where, H is the entropy in the system which is a measure of randomness.

Assuming you are rolling a fair coin and want to know the Entropy of the system. As per the formula given by Shann – Entropy would be equals to $-[0.5 \ln(0.5) + 0.5 \ln(0.5)]$.

Which is equal to -0.69; which is the maximum entropy which can occur in the system. In other words, there will be maximum randomness in our dataset if the probable outcomes have same probability of occurrence.



Graph shown above shows the variation of Entropy with the probability of a class, we can clearly see that Entropy is maximum when probability of either of the classes is equal. Now, you can understand that when a decision algorithm tries to split the data, it selects the variable which will give us maximum reduction in system Entropy.

For the example of movie success rate – Initial Entropy in the system was:

$Entropy_{Parent} = -(0.57 * \ln(0.57) + 0.43 * \ln(0.43))$; Which is 0.68

Entropy after Method 1 Split

$Entropy_{left} = -(.75 * \ln(0.75)) + 0.25 * \ln(0.25) =$

$Entropy_{right} = -(0.33 * \ln(0.33) + 0.67 * \ln(0.67)) = 0.63$

Captured impurity or entropy after splitting data using Method 1 can be calculated using the following formula: “Entropy (Parent) – Weighted Average of Children Entropy”

Which is,

$$0.68 - (4 * 0.56 + 3 * 0.63) / 7 = 0.09$$

This number 0.09 is generally known as “Information Gain”

Entropy after Method 2 Split

$Entropy_{left} = -(.67 * \ln(0.67)) + 0.33 * \ln(0.33) =$

$Entropy_{middle} = -(0.5 * \ln(0.5)) + 0.5 * \ln(0.5) =$

$Entropy_{right} = -(0.5 * \ln(0.5) + 0.5 * \ln(0.5)) = 0.69$

Now using the method used above, we can calculate the Information Gain as:

$$\text{Information Gain} = 0.68 - (3 \cdot 0.63 + 2 \cdot 0.69 + 2 \cdot 0.69) / 7 = 0.02$$

Hence, we can clearly see that Method 1 gives us more than 4 times information gain compared to Method 2 and hence Method 1 is the best split variable.

Gain Ratio

Soon after the development of entropy mathematicians realized that Information gain is biased toward multi-valued attributes and to conquer this issue, “Gain Ratio” came into picture which is more reliable than Information gain. The gain ratio can be defined as:

$$\text{Gini}_S(K) = \frac{T_1}{T} \text{Gini}(T_1) + \frac{T_2}{T} \text{Gini}(T_2)$$

Where Split info can be defined as:

$$-\sum_{i=1}^n D_i \log_2 D_i$$

Assuming we are dividing our variable into ‘n’ child nodes and D_i represents the number of records going into various child nodes. Hence gain ratio takes care of distribution bias while building a decision tree.

For the example discussed above, for Method 1

$$\text{Split Info} = -((4/7) \cdot \log_2(4/7)) - ((3/7) \cdot \log_2(3/7)) = 0.98$$

And Hence,

$$\text{Gain Ratio} = 0.09 / 0.98 = 0.092$$

Gini Index

There is one more metric which can be used while building a decision tree is Gini Index (Gini Index is mostly used in CART). Gini index measures the impurity of a data partition K, formula for Gini Index can be written down as:

$$\text{Gini}(K) = 1 - \sum_{i=1}^n P_i^2$$

Where m is the number of classes, and P_i is the probability that an observation in K belongs to the class. Gini Index assumes a binary split for each of the attribute in S, let say T_1 & T_2 . The Gini index of K given this partitioning is given by:

Which is nothing but a weighted sum of each of the impurities in split nodes. The reduction in impurity is given by:

$$\text{Gini}(K) - \text{Gini}_S(K)$$

Similar to Information Gain & Gain Ratio, split which gives us maximum reduction in impurity is considered for dividing our data.

Coming back to our movie example, If we want to calculate $Gini(K)$ -

$$= 0.49$$

Now as per our Method 1, we can get $Gini_S(K)$ as,

$$\begin{aligned} Gini_S(K) &= \frac{T_1}{T} Gini(T_1) + \frac{T_2}{T} Gini(T_2) \\ &= \frac{4}{7} \left\{ 1 - \left(\left[\frac{3}{4} \right]^2 + \left[\frac{1}{4} \right]^2 \right) \right\} + \frac{3}{7} \left\{ 1 - \left(\left[\frac{1}{3} \right]^2 + \left[\frac{2}{3} \right]^2 \right) \right\} \\ Gini(K) &= 1 - \left(\left[\frac{4}{7} \right]^2 + \left[\frac{3}{7} \right]^2 \right) \\ &= 0.24 + 0.19 \\ &= 0.43 \end{aligned}$$

LINEAR REGRESSION

Linear regression is a statistical approach for modelling relationship between a dependent variable with a given set of independent variables.

Simple Linear Regression

Simple linear regression is an approach for predicting a **response** using a **single feature**.

It is assumed that the two variables are linearly related. Hence, we try to find a linear function that predicts the response value (y) as accurately as possible as a function of the feature or independent variable (x).

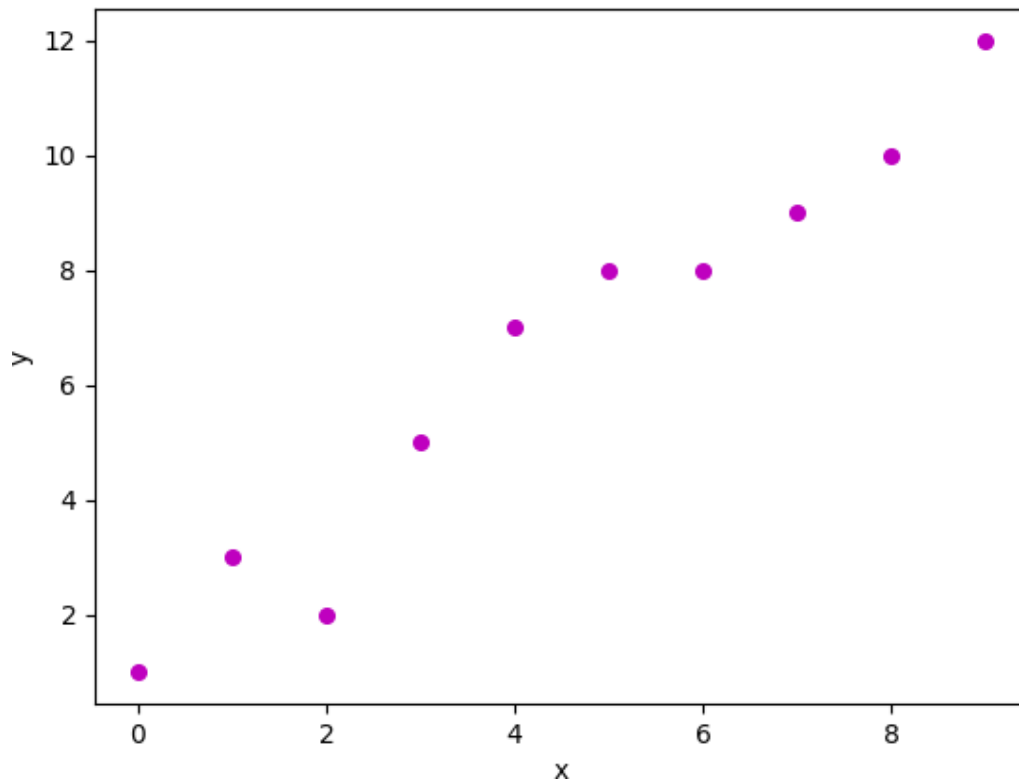
Let us consider a dataset where we have a value of response y for every feature x :

x	0	1	2	3	4	5	6	7	8	9
y	1	3	2	5	7	8	8	9	10	12

For generality, we define:

x as **feature vector**, i.e $x = [x_1, x_2, \dots, x_n]$, y as **response vector**, i.e $y = [y_1, y_2, \dots, y_n]$ for n observations (in above example, $n=10$).

A scatter plot of above dataset looks like:-



Now, the task is to find a **line which fits best** in above scatter plot so that we can predict the response for any new feature values. (i.e a value of x not present in dataset)

This line is called **regression line**.

The equation of regression line is represented as:

Here,

- $h(x_i)$ represents the **predicted response value** for i th observation.
- b_0 and b_1 are regression coefficients and represent **y-intercept** and **slope** of regression line respectively.

To create our model, we must “learn” or estimate the values of regression coefficients b_0 and b_1 . And once we’ve estimated these coefficients, we can use the model to predict responses!

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i = h(x_i) + \varepsilon_i \Rightarrow \varepsilon_i = y_i - h(x_i)$$

Here, ε_i is **residual error** in i th observation.

So, our aim is to minimize the total residual error.

We define the squared error or cost function, J as:

$$J(\beta_0, \beta_1) = \frac{1}{2n} \sum_{i=1}^n \varepsilon_i^2$$

and our task is to find the value of β_0 and β_1 for which $J(\beta_0, \beta_1)$ is minimum!

Without going into the mathematical details, we present the result here:

$$\beta_1 = \frac{SS_{xy}}{SS_{xx}}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

where SS_{xy} is the sum of cross-deviations of y and x :

$$SS_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \sum_{i=1}^n y_i x_i - n\bar{x}\bar{y}$$

and SS_{xx} is the sum of squared deviations of x :

$$SS_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n x_i^2 - n(\bar{x})^2$$

LOGISTIC REGRESSION

Consider a model with features $x_1, x_2, x_3 \dots x_n$. Let the binary output be denoted by Y , that can take the values 0 or 1.

Let p be the probability of $Y = 1$, we can denote it as $p = P(Y=1)$.

The mathematical relationship between these variables can be denoted as:

$$\ln\left(\frac{p}{1-p}\right) = b_0 + b_1 x_1 + b_2 x_2 + b_3 x_3 \dots b_n x_n$$

Here the term $p/(1-p)$ is known as the *odds* and denotes the likelihood of the event taking place. Thus $\ln(p/(1-p))$ is known as the *log odds* and is simply used to map the probability that lies between 0 and 1 to a range between $(-\infty, +\infty)$. The terms $b_0, b_1, b_2 \dots$ are parameters (or weights) that we will estimate during training.

So this is just the basic math behind what we are going to do. We are interested in the probability p in this equation. So we simplify the equation to obtain the value of p :

1. The log term $\ln$ on the LHS can be removed by raising the RHS as a power of e :

$$\frac{p}{1-p} = e^{b_0+b_1x_1+b_2x_2+b_3x_3\dots b_nx_n}$$

2. Now we can easily simplify to obtain the value of p :

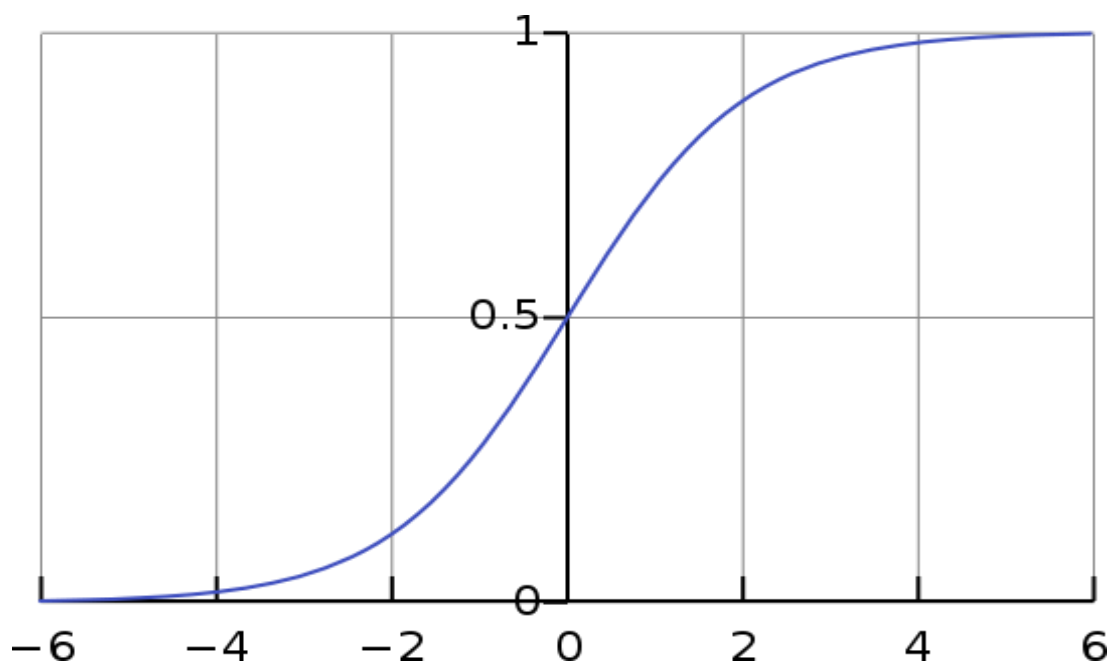
$$p = \frac{e^{b_0+b_1x_1+b_2x_2+b_3x_3\dots b_nx_n}}{1 + e^{b_0+b_1x_1+b_2x_2+b_3x_3\dots b_nx_n}}$$

or

$$p = \frac{1}{1 + e^{-(b_0+b_1x_1+b_2x_2+b_3x_3\dots b_nx_n)}}$$

This actually turns out to be the equation of the *Sigmoid Function* which is widely used in other machine learning applications. The *Sigmoid Function* is given by:

$$S(x) = \frac{1}{1 + e^{-x}}$$



The sigmoid curve

Now we will be using the above derived equation to make our predictions. Before that we will train our model to obtain the values of our parameters $b_0, b_1, b_2 \dots$ that result in least error. This is where the error or loss function comes in.

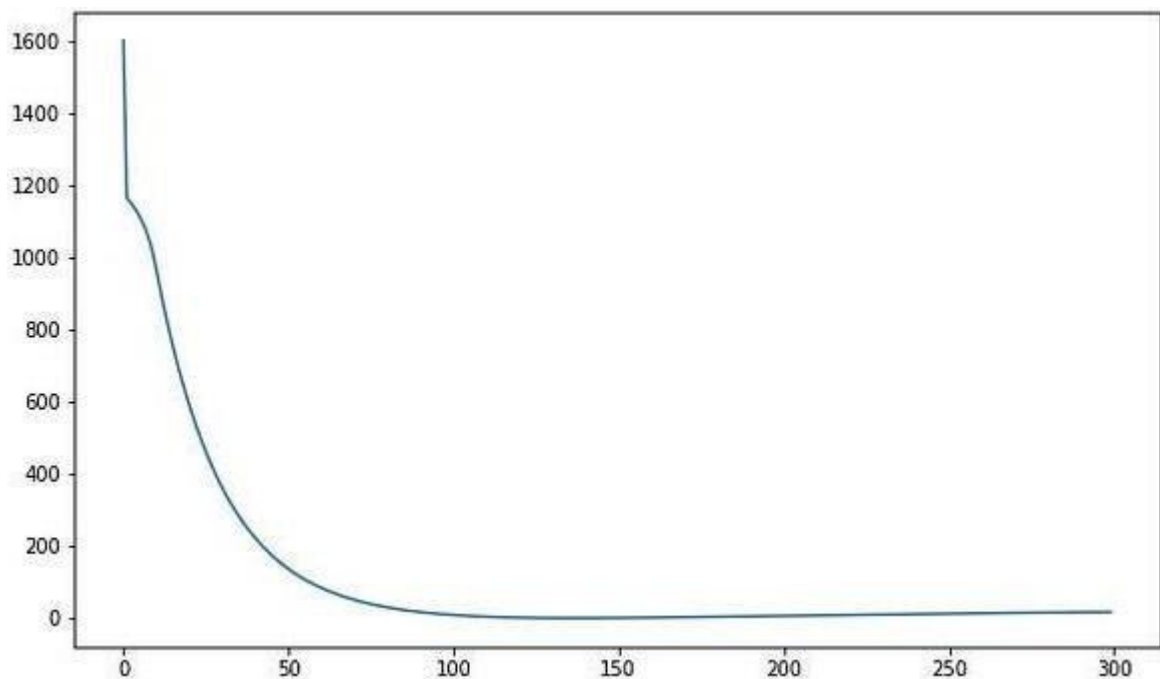
Loss Function

The loss is basically the error in our predicted value. In other words it is a difference between our predicted value and the actual value. We will be using the L2 Loss Function to calculate the error. Theoretically you can use any function to calculate the error. This function can be broken down as:

1. Let the actual value be y_i . Let the value predicted using our model be denoted as $\bar{y}_i$. Find the difference between the actual and predicted value.
2. Square this difference.
3. Find the sum across all the values in training data.

$$L = \sum_{i=1}^n (y_i - \bar{y}_i)^2$$

Now that we have the error, we need to update the values of our parameters to minimize this error. This is where the “learning” actually happens, since our model is updating itself based on it’s previous output to obtain a more accurate output in the next step. Hence with each iteration our model becomes more and more accurate. We will be using the *Gradient Descent Algorithm* to estimate our parameters. Another commonly used algorithm is the Maximum Likelihood Estimation.



The loss or error on the y axis and number of iterations on the x axis.

The Gradient Descent Algorithm

You might know that the partial derivative of a function at it's minimum value is equal to 0. So gradient descent basically uses this concept to estimate the parameters or weights of our model by minimizing the loss function. [Click here](#) for a more detailed explanation on how gradient descent works. For simplicity, for the rest of this tutorial let us assume that our output depends only on a single feature x . So we can rewrite our equation as:

$$\bar{y}_i = p = \frac{1}{1 + e^{-(b_0 + b_1 x_i)}} = \frac{1}{1 + e^{-b_0 - b_1 x_i}}$$

Thus we need to estimate the values of weights b_0 and b_1 using our given training data.

1. Initially let $b_0=0$ and $b_1=0$. Let L be the learning rate. The learning rate controls by how much the values of b_0 and b_1 are updated at each step in the learning process. Here let $L=0.001$.

2. Calculate the partial derivative with respect to b_0 and b_1 . The value of the partial derivative will tell us how far the loss function is from its minimum value. It is a measure of how much our weights need to be updated to attain minimum or ideally 0 error. In case you have more than one feature, you need to calculate the partial derivative for each weight $b_0, b_1 \dots b_n$ where n is the number of features. For a detailed explanation on the math behind calculating the partial derivatives.

$$D_{b_0} = -2 \sum_{i=1}^n (y_i - \bar{y}_i) \times \bar{y}_i \times (1 - \bar{y}_i)$$

$$D_{b_1} = -2 \sum_{i=1}^n (y_i - \bar{y}_i) \times \bar{y}_i \times (1 - \bar{y}_i) \times x_i$$

3. Next we update the values of b_0 and b_1 :

$$b_0 = b_0 - L \times D_{b_0}$$

$$b_1 = b_1 - L \times D_{b_1}$$

4. We repeat this process until our loss function is a very small value or ideally reaches 0 (meaning no errors and 100% accuracy). The number of times we repeat this learning process is known as iterations or epochs.

Differences between Linear Regression and Logistic Regression:-**LINEAR REGRESSION**

Linear Regression is a supervised regression

model.

supervised classification model.

In Linear Regression, we predict the value by

an integer number.

predict the value by 1 or 0.

Here no activation function is used.

to convert a linear regression

LOGISTIC REGRESSION

Logistic Regression is a

In Logistic Regression, we

Here activation function is used

Generalized Linear Models:-

The Generalized Linear Model (GLM) is a generalization of the general linear model. In its simplest form, a linear model specifies the (linear) relationship between a dependent (or response) variable Y , and a set of predictor variables, the X 's, so that

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_kX_k$$

In this equation b_0 is the regression coefficient for the intercept and the b_i values are the regression coefficients (for variables 1 through k) computed from the data.

So for example, we could estimate (i.e., predict) a person's weight as a function of the person's height and gender. You could use linear regression to estimate the respective regression coefficients from a sample of data, measuring height, weight, and observing the subjects' gender. For many data analysis problems, estimates of the linear relationships between variables are adequate to describe the observed data, and to make reasonable predictions for new observations..

However, there are many relationships that cannot adequately be summarized by a simple linear equation, for two major reasons:

Distribution of dependent variable. First, the dependent variable of interest may have a non-continuous distribution, and thus, the predicted values should also follow the respective distribution; any other predicted values are not logically possible. For example, a researcher may be interested in predicting one of three possible discrete outcomes (e.g., a consumer's choice of one of three alternative products). In that case, the dependent variable can only take on 3 distinct values, and the distribution of the dependent variable is said to be multinomial. Or suppose you are trying to predict people's family planning choices, specifically, how many children families will have, as a function of income and various other socioeconomic indicators. The

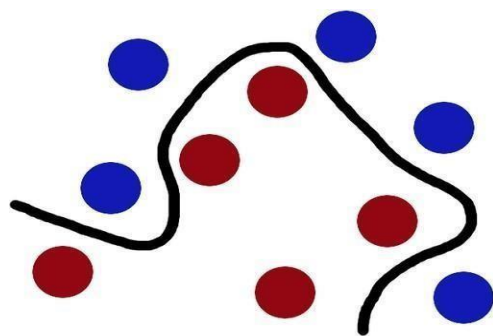
dependent variable - number of children - is discrete (i.e., a family may have 1, 2, or 3 children and so on, but cannot have 2.4 children), and most likely the distribution of that variable is highly skewed (i.e., most families have 1, 2, or 3 children, fewer will have 4 or 5, very few will have 6)

SUPPORT VECTOR MACHINES

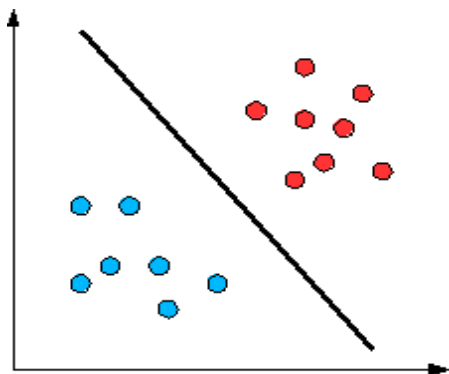
Support Vector Machine or SVM are supervised learning models with associated learning [algorithms](#) that analyze data for classification (classification means knowing what belongs to what e.g. 'apple' belongs to class 'fruit' while 'dog' to class 'animals' -see fig.1)

In support vector machines, it looks somewhat like which separates the blue balls from red.

SVM is a **classifier** formally **defined** by a separating hyperplane. An hyperplane is a subspace of one **dimension** less than its **ambient space**. The **dimension** of a mathematical space (or object) is informally defined as the minimum number of coordinates (x,y,z axis) needed to specify any point (like each blue and red point) within it while an ambient space is the space surrounding a mathematical object.



Therefore the hyperplane of a two dimensional space below (fig.2) is a one dimensional line dividing the red and blue dots.

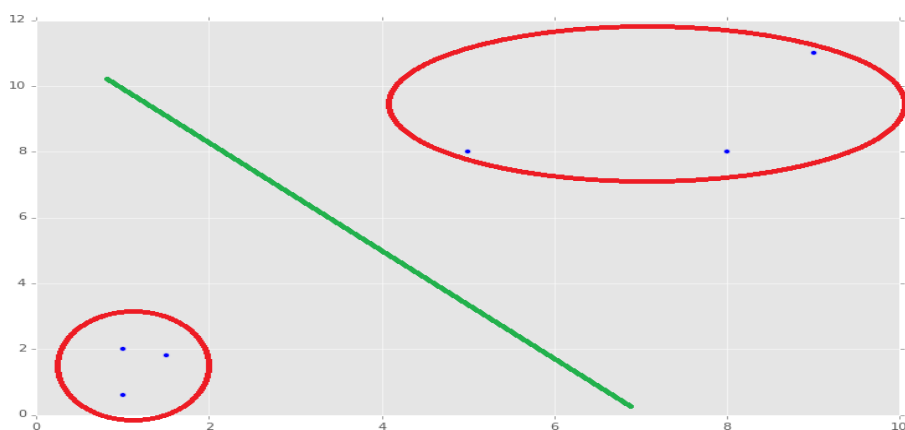


Can you try to solve the above problem linearly like we did with Fig. 2? NO!

The red and blue balls cannot be separated by a straight line as they are randomly distributed and this, in reality, is how most real life problem data are **-randomly distributed**.

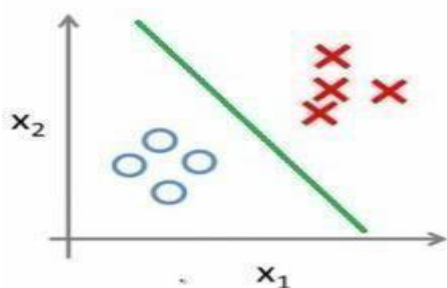
In machine learning, a “kernel” is usually used to refer to the kernel trick, a method of using a linear classifier to solve a non-linear problem. It entails transforming linearly inseparable data like (Fig. 3) to linearly separable ones (Fig. 2). The kernel function is what is applied on each data instance to map the original non-linear observations into a higher-dimensional space in which they become separable.

Using the dog breed prediction example again, kernels offer a better alternative. Instead of defining a slew of features, you define a single kernel function to compute **similarity** between breeds of dog. You provide this kernel, together with the data and labels to the learning algorithm, and out comes a classifier.

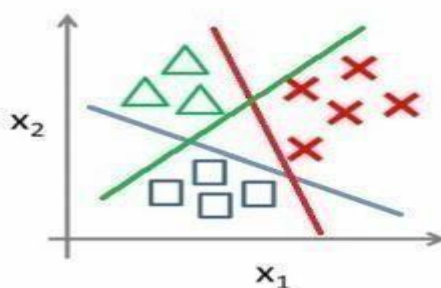


So this is with two features, and we see we have a 2D graph. If we had three features, we could have a 3D graph. The 3D graph would be a little more challenging for us to visually group and divide, but still do-able. The problem occurs when we have four features, or four-thousand features. Now you can start to understand the power of machine learning, seeing and analyzing a number of dimensions imperceptible to us.

Binary classification:



Multi-class classification:



Common examples include **image classification** (is it a cat, dog, human, etc) or **handwritten digit recognition** (classifying an image of a handwritten number into a digit from 0 to 9).

This algorithm applies the same trick as k-means but with one difference that here in the calculation of distance, kernel method is used instead of the Euclidean distance.

UNIT-III

Supervised Learning – II (Neural Networks)

Neural Network Representation – Problems – Perceptrons , Activation Functions, Artificial Neural Networks(ANN) , Back Propagation Algorithm.

Convolutional Neural Networks - Convolution and Pooling layers, , Recurrent Neural Networks (RNN).
Classification Metrics: Confusion matrix, Precision, Recall, Accuracy, F-Score, ROC curves

ARTIFICIAL NEURON MODEL

An artificial neuron is a mathematical function conceived as a simple model of a real(biological) neuron.

- The McCulloch-Pitts Neuron

This is a simplified model of real neurons, known as a Threshold Logic Unit.

- A set of input connections brings in activations from other neuron.
- A processing unit sums the inputs, and then applies a non-linear activation function(i.e. squashing/transfer/threshold function).
- An output line transmits the result to other neurons.

1.2.1 Basic Elements of ANN

Neuron consists of three basic components –weights, thresholds and a single activationfunction. An Artificial neural network(ANN) model based on the biological neural sytems is shown in Figure 2.

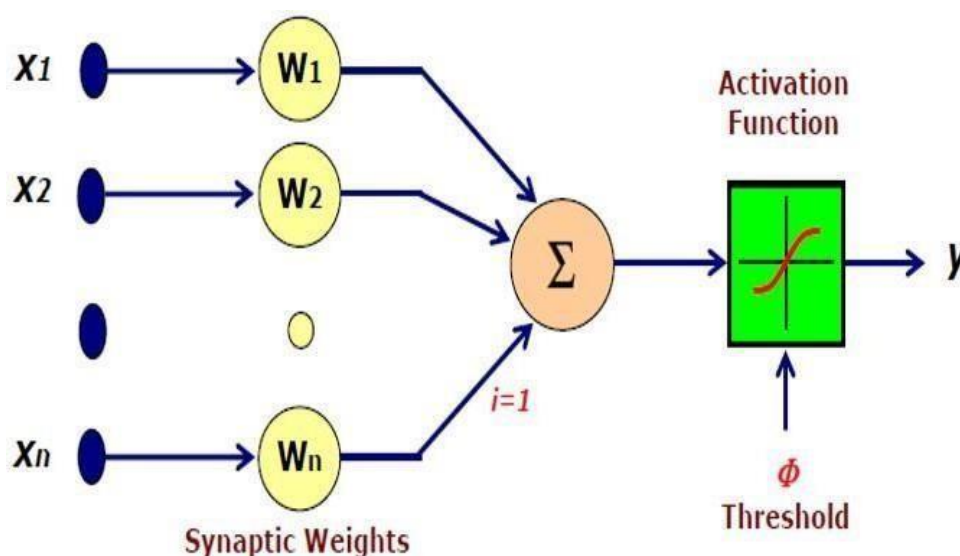


Figure 2: Basic Elements of Artificial Neural Network

DIFFERENT LEARNING RULES

A brief classification of Different Learning algorithms is depicted in figure 3.

- Training: It is the process in which the network is taught to change its weight and bias.
- Learning: It is the internal process of training where the artificial neural system learns to update/adapt the weights and biases.

Different Training /Learning procedure available in ANN are

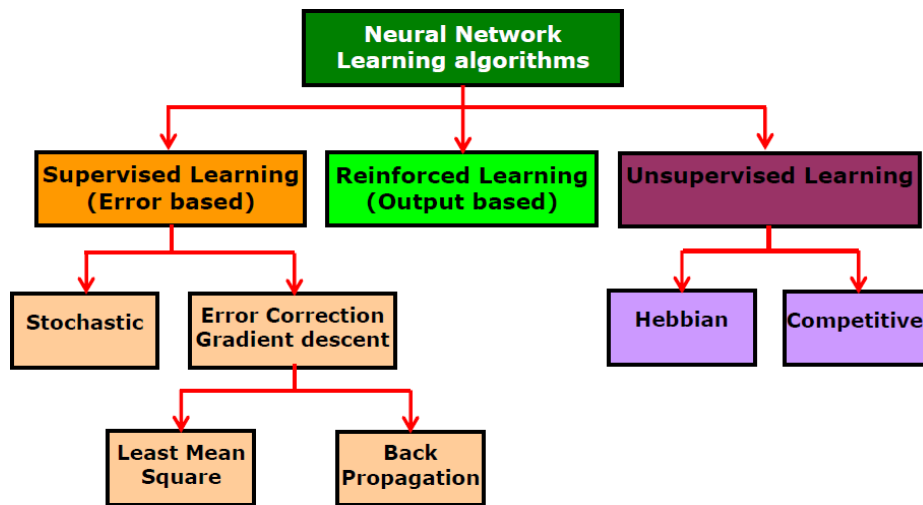
Supervised learning

- Unsupervised learning
- Reinforced learning
- Hebbian learning
- Gradient descent learning
- Competitive learning
- Stochastic learning

Requirements of Learning Laws

- Learning Law should lead to convergence of weights
- Learning or training time should be less for capturing the information from the training pairs
- Learning should use the local information
- Learning process should be able to capture the complex non linear mapping available between the input & output pairs
- Learning should be able to capture as many as patterns as possible
- Storage of pattern information's gathered at the time of learning should be high for the given network

Figure 3: Different Training methods of Artificial Neural Network



ANN Back Propagation;

Backpropagation is a widely used algorithm for training feedforward neural networks. It computes the gradient of the loss function with respect to the network weights. It is very efficient, rather than naively directly computing the gradient concerning each weight. This efficiency makes it possible to use gradient methods to train multi-layer networks and update weights to minimize loss; variants such as gradient descent or stochastic gradient descent are often used.

The backpropagation algorithm works by computing the gradient of the loss function with respect to each weight via the chain rule, computing the gradient layer by layer, and iterating backward from the last layer to avoid redundant computation of intermediate terms in the chain rule.

Features of Backpropagation:

1. it is the [gradient descent](#) method as used in the case of simple perceptron network with the differentiable unit.
2. it is different from other networks in respect to the process by which the weights are calculated during the learning period of the network.
3. training is done in the three stages :
 - the [feed-forward](#) of input training pattern
 - the calculation and backpropagation of the error
 - updation of the weight

Backpropagation Algorithm:

Step 1: Inputs X, arrive through the preconnected path.

Step 2: The input is modeled using true weights W. Weights are usually chosen randomly.

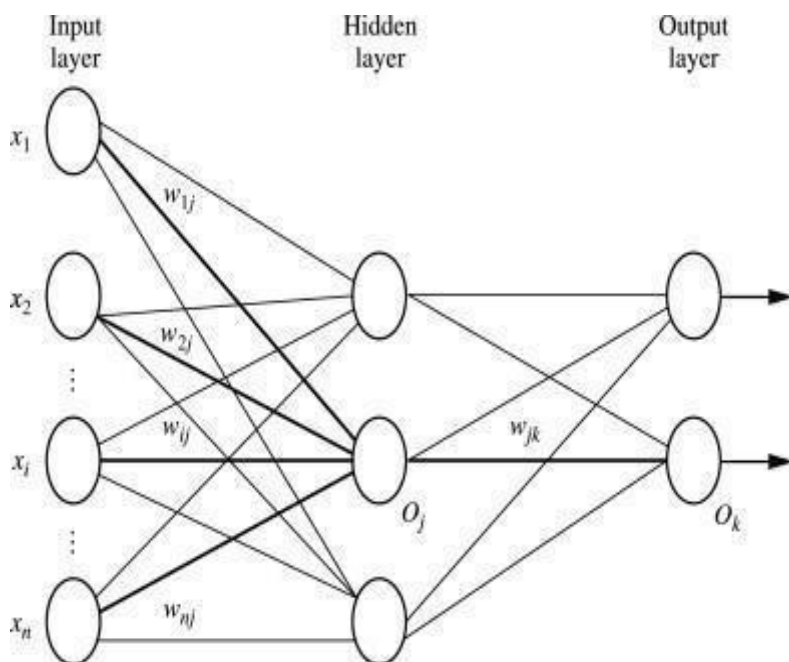
Step 3: Calculate the output of each neuron from the input layer to the hidden layer to the output layer.

Step 4: Calculate the error in the outputs

Backpropagation Error= Actual Output – Desired Output

Step 5: From the output layer, go back to the hidden layer to adjust the weights to reduce the error.

Step 6: Repeat the process until the desired output is achieved.



Parameters :

- x = inputs training vector $x=(x_1, x_2, \dots, x_n)$.
- t = target vector $t=(t_1, t_2, \dots, t_n)$.
- δ_k = error at output unit.
- δ_j = error at hidden layer.
- α = learning rate.
- V_{0j} = bias of hidden unit j .

Need for Backpropagation:

Backpropagation is “backpropagation of errors” and is very useful for training neural networks. It’s fast, easy to implement, and simple. Backpropagation does not require any parameters to be set, except the number of inputs. Backpropagation is a flexible method because no prior knowledge of the network is required.

Types of Backpropagation

There are two types of backpropagation networks.

- **Static backpropagation:** Static backpropagation is a network designed to map static inputs for static outputs. These types of networks are capable of solving static classification problems such as OCR (Optical Character Recognition).
- **Recurrent backpropagation:** Recursive backpropagation is another network used for fixed-point learning. Activation in recurrent backpropagation is feed-forward until a fixed value is reached. Static backpropagation provides an instant mapping, while recurrent backpropagation does not provide an instant mapping.

Advantages:

- It is simple, fast, and easy to program.
- Only numbers of the input are tuned, not any other parameter.
- It is Flexible and efficient.
- No need for users to learn any special functions.

Disadvantages:

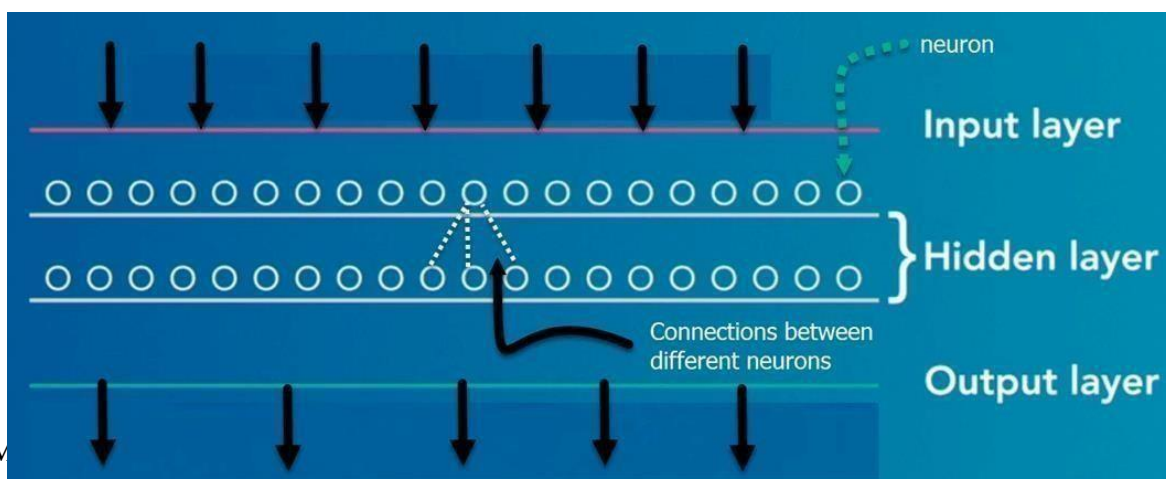
- It is sensitive to noisy data and irregularities. Noisy data can lead to inaccurate results.
- Performance is highly dependent on input data.
- Spending too much time training.
- The matrix-based approach is preferred over a mini-batch.

What is Deep Learning?

Deep learning is a computer software that **mimics the network of neurons in a brain**. It is a subset of machine learning and is called deep learning because it makes use of deep **neural networks**.

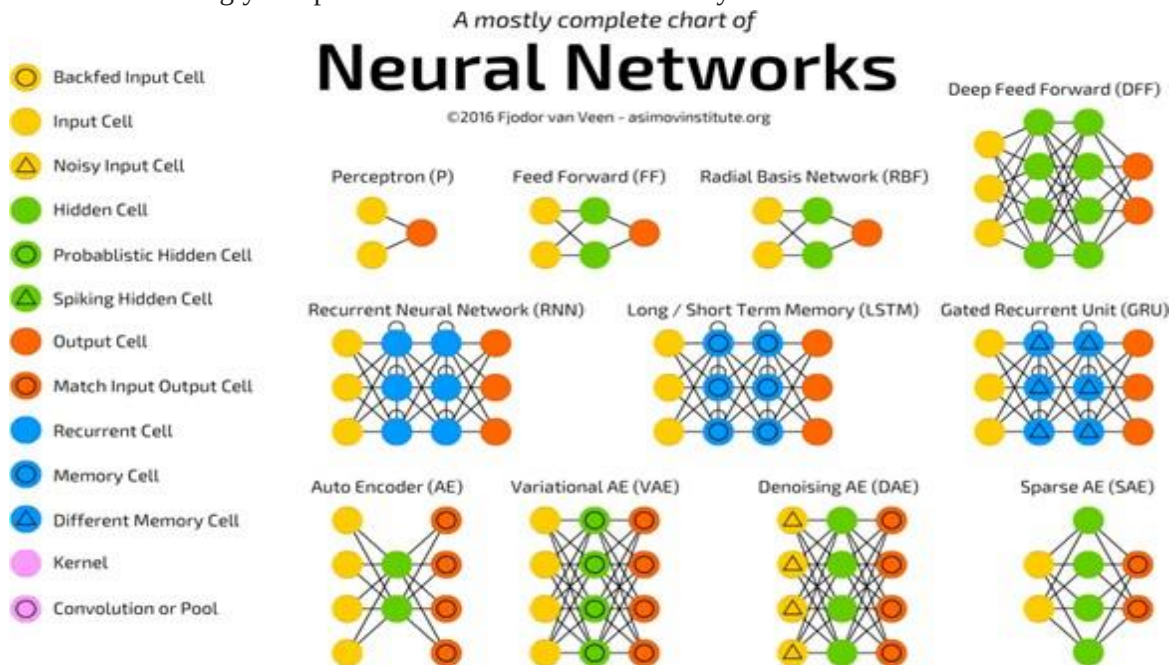
Deep learning algorithms are constructed with connected layers.

- The first layer is called the Input Layer
- The last layer is called the Output Layer
- All layers in between are called Hidden Layers. The word deep means the network join neurons in more than two layers.



Each Hidden layer is composed of neurons. The neurons are connected to each other. The neuron will process and then propagate the input signal it receives the layer above it. The strength of the signal given the neuron in the next layer depends on the weight, bias and activation function.

The network consumes large amounts of input data and operates them through multiple layers; the network can learn increasingly complex features of the data at each layer.



Why is Deep Learning Important?

Deep learning is a powerful tool to make prediction an actionable result. Deep learning excels in pattern discovery (unsupervised learning) and knowledge-based prediction. Big data is the fuel for deep learning. When both are combined, an organization can reap unprecedented results in term of productivity, sales, management, and innovation.

Deep learning can outperform traditional method. For instance, deep learning algorithms are 41% more accurate than machine learning algorithm in image classification, 27 % more accurate in facial recognition and 25% in voice recognition.

Limitations of deep learning

Data labelling
Most current AI models are trained through "supervised learning." It means that humans must label and categorize the underlying data, which can be a sizable and error-prone chore. For example, companies developing self-driving- car technologies are hiring hundreds of people to manually annotate hours of video feeds from prototype vehicles to help train these systems.

Obtain huge training datasets

It has been shown that simple deep learning techniques like CNN can, in some cases, imitate the knowledge of experts in medicine and other fields. The current wave of machine learning, however, requires training data sets that are not only labeled but also sufficiently broad and universal.

Deep-learning methods required thousands of observation for models to become relatively good at classification tasks and, in some cases, millions for them to perform at the level of humans. Without surprise, deep learning is famous in giant tech companies; they are using big data to accumulate petabytes of data. It

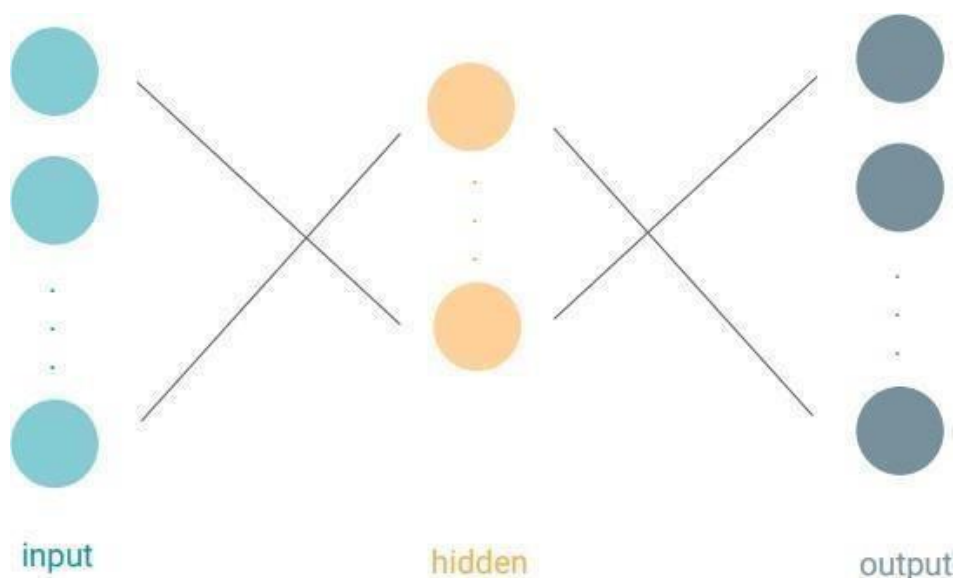
allows them to create an impressive and highly accurate deep learning model.

Unsupervised

Unsupervised feature learning is learning features from unlabeled data. The goal of unsupervised feature learning is often to discover low-dimensional features that captures some structure underlying the high-dimensional input data. When the feature learning is performed in an unsupervised way, it enables a form of semisupervised learning where features learned from an unlabeled dataset are then employed to improve performance in a supervised setting with labeled data. Several approaches are introduced in the following.

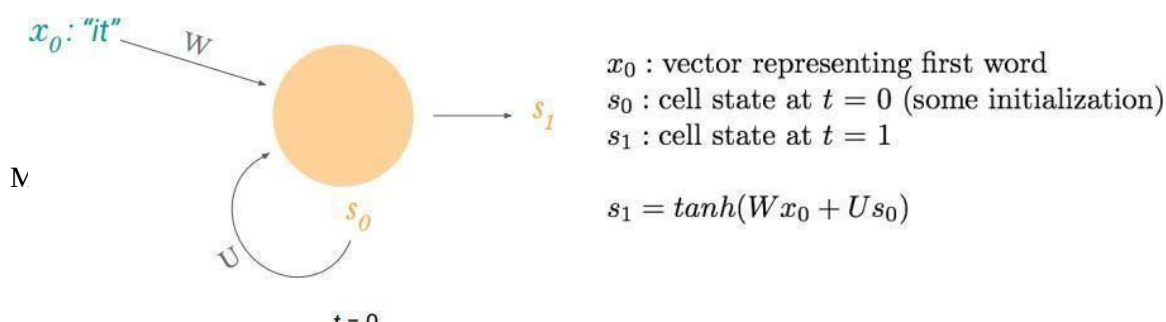
Recurrent Neural Network

A Recurrent Neural Network is architected in the same way as a “traditional” Neural Network. We have some inputs, we have some hidden layers and we have some outputs.



The only difference is that each hidden unit is doing a slightly different function. So, let's explore how this hidden unit works.

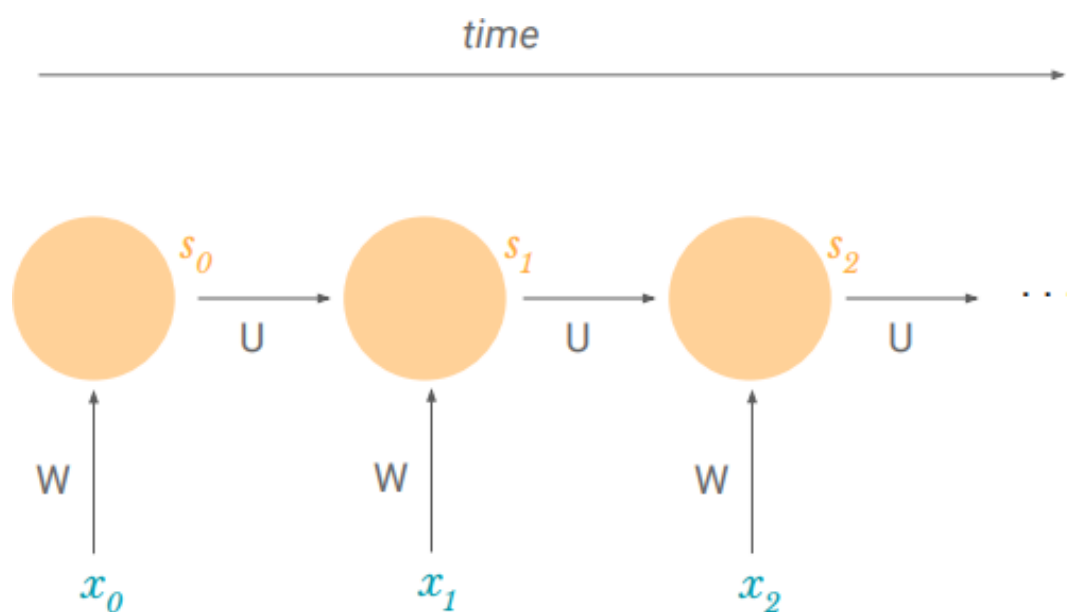
A recurrent hidden unit computes a function of an input and its own previous output, also known as the cell state. For textual data, an input could be a vector representing a word $x(i)$ in a sentence of n words (also known as word embedding).



W and U are weight matrices and $\tanh$ is the hyperbolic tangent function.

Similarly, at the next step, it computes a function of the new input and its previous cell state: $s_2 = \tanh(Wx_1 + Us_1)$. This behavior is similar to a hidden unit in a feed-forward Network. The difference, proper to sequences, is that we are adding an additional term to incorporate its own previous state.

A common way of viewing recurrent neural networks is by unfolding them across time. We can notice that **we are using the same weight matrices W and U throughout the sequence. This solves our problem of parameter sharing.** We don't have new parameters for every point of the sequence. Thus, once we learn something, it can apply at any point in the sequence.



The fact of not having new parameters for every point of the sequence also helps us **deal with variable-length sequences**. In case of a sequence that has a length of 4, we could unroll this RNN to four timesteps. In other cases, we can unroll it to ten timesteps since the length of the sequence is not prespecified in the algorithm. By unrolling we simply mean that we write out the network for the complete sequence. For example, if the sequence we care about is a sentence of 5 words, the network would be unrolled into a 5-layer neural network, one layer for each word.

Introduction to Convolution Neural Network

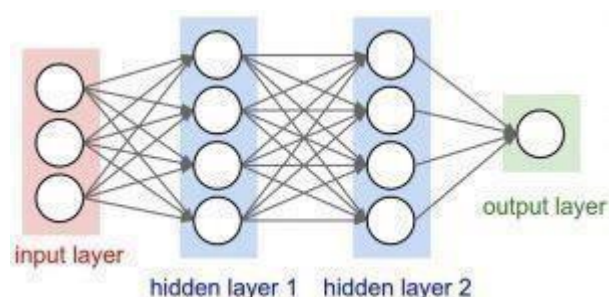
It is assumed that the reader knows the concept of Neural networks. When it comes to Machine Learning, [Artificial Neural Networks](#) perform really well. Artificial Neural Networks are used in various classification tasks like image, audio, words. Different types of Neural Networks are used for different purposes, for example for predicting the sequence of words we use Recurrent Neural Networks more precisely an LSTM, similarly for image classification we use Convolution Neural networks. In this blog, we are going to build a basic building block regular Neural Network there are three types of

1. **Input Layers:** It's the layer in which we give input to our model. The number of neurons in this layer is equal to the total number of features in our data (number of pixels in the case of an image).
2. **Hidden Layer:** The input from the Input layer is then feed into the hidden layer. There can be many hidden layers depending upon our model and data size. Each hidden layer can have different numbers of neurons which are generally greater than the number of features. The output from each layer is computed by matrix multiplication of output of the previous layer with learnable weights of that layer and then by the addition of learnable biases followed by activation function which makes the network nonlinear.
3. **Output Layer:** The output from the hidden layer is then fed into a logistic function like sigmoid or softmax which converts the output of each class into the probability score of each class.

The data is then fed into the model and output from each layer is obtained this step is called feedforward, we then calculate the error using an error function, some common error functions are cross-entropy, square loss error, etc. After that, we backpropagate into the model by calculating the derivatives. This step is called Back propagation which basically is used to minimize the loss. Here's the basic python code for a neural network with random inputs and two hidden layers.

```
activation = lambda x: 1.0/(1.0 + np.exp(-x)) # sigmoid function
input = np.random.randn(3, 1)
hidden_1 = activation(np.dot(W1, input) + b1)
hidden_2 = activation(np.dot(W2, hidden_1) + b2)
output = np.dot(W3, hidden_2) + b3
```

$W1, W2, W3, b1, b2, b3$ are learnable parameter of the model.

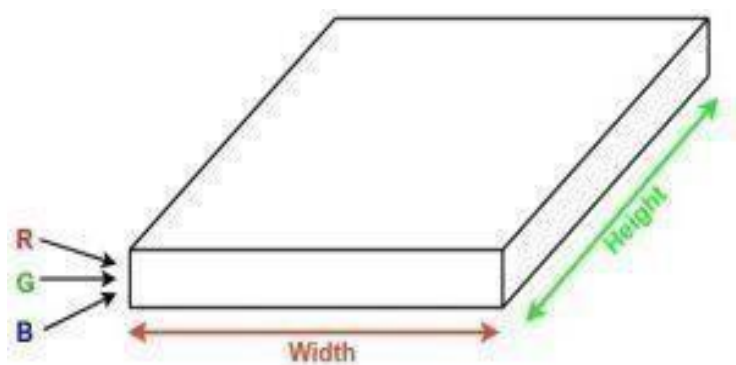


Convolution Neural Network

Convolution Neural Networks or convnets are neural networks that share their parameters. Imagine you have an image. It can be represented as a cuboid having its length, width (dimension of the image), and height (as

Images generally have red, green, and blue channels).

Now imagine taking a small patch of this image and running a small neural network on it, with say, k outputs and represent them vertically. Now slide that neural network across the whole image, as a result, we will get another image with different width, height, and depth. Instead of just R, G, and B channels now we have more channels but lesser width and height. This operation is called Convolution. If the patch size is the same as that of the image it will be a regular neural network. Because of this small patch, we have fewer weights.



Now let's talk about a bit of mathematics that is involved in the whole convolution process.

- Convolution layers consist of a set of learnable filters (a patch in the above image). Every filter has small width and height and the same depth as that of input volume (3 if the input layer is image input).
- For example, if we have to run convolution on an image with dimension $34 \times 34 \times 3$. The possible size of filters can be $a \times a \times 3$, where 'a' can be 3, 5, 7, etc but small as compared to image dimension.
- During forward pass, we slide each filter across the whole input volume step by step where each step is called stride (which can have value 2 or 3 or even 4 for high dimensional images) and compute the dot product between the weights of filters and patch from input volume.
- As we slide our filters we'll get a 2-D output for each filter and we'll stack them together and as a result, we'll get output volume having a depth equal to the number of filters. The network will learn all the filters.

Layers used to build ConvNets

A convnets is a sequence of layers, and every layer transforms one volume to another through a differentiable function.

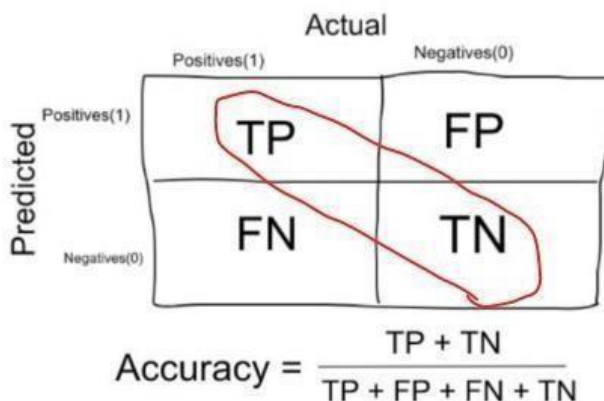
Types of layers:

1. **Input Layer:** This layer holds the raw input of the image with width 32, height 32, and depth 3.
2. **Convolution Layer:** This layer computes the output volume by computing the dot product between all filters and image patches. Suppose we use a total of 12 filters for this layer we'll get output volume of dimension $32 \times 32 \times 12$.
3. **Activation Function Layer:** This layer will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are RELU: $\max(0, x)$, Sigmoid: $1/(1+e^{-x})$, Tanh, Leaky RELU, etc. The volume remains unchanged hence output volume will have dimension $32 \times 32 \times 12$.
4. **Pool Layer:** This layer is periodically inserted in the convnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are **max pooling** and **average pooling**. If we use a max pool with 2×2 filters and stride 2, the resultant volume will be of dimension .

Performance Metrics

- **Accuracy** can be calculated by taking average of the values lying across the “main diagonal” i.e

$$\text{Accuracy} = (\text{True Positives} + \text{False Negatives}) / \text{Total Number of Samples}$$



Precision: -It is the number of correct positive results divided by the number of positive results predicted by classifier.

		Actual	
		Positives(1)	Negatives(0)
Predicted	Positives(1)	TP	FP
	Negatives(0)	FN	TN

Precision = $\frac{TP}{TP + FP}$

- Recall :- It is the number of correct positive results divided by the number of all relevant samples

		Actual	
		Positives(1)	Negatives(0)
Predicted	Positives(1)	TP	FP
	Negatives(0)	FN	TN

Recall = $\frac{TP}{TP + FN}$

Structured prediction or structured (output) learning :-

It is an umbrella term for supervised machine learning techniques that involves predicting structured objects, rather than scalar discrete or real values.

Similar to commonly used supervised learning techniques, structured prediction models are typically trained by means of observed data in which the true prediction value is used to adjust model parameters. Due to the complexity of the model and the interrelations of predicted variables the process of prediction using a trained model and of training itself is often computationally infeasible and approximate inference and learning methods are used.

For example, the problem of translating a natural language sentence into a syntactic representation such as a parse tree can be seen as a structured prediction problem in which the structured output domain is the set of all possible parse trees. Structured prediction is also used in a wide variety of application domains including bioinformatics, natural language processing, speech recognition, and computer vision.

Example: sequence tagging

Sequence tagging is a class of problems prevalent in natural language processing, where input data are often sequences (e.g. sentences of text). The sequence tagging problem appears in several guises, e.g. part-of-speech tagging and named entity recognition. In POS tagging, for example, each word in a sequence must receive a "tag" (class label) that expresses its "type" of word:

DT-Determiner VB-Verb
JJ-Adjective NN-Noun

Ranking :-

Learning to Rank (LTR) is a class of techniques that apply supervised machine learning (ML) to solve **ranking problems**. The main difference between LTR and traditional supervised ML is this:

- Traditional ML solves a prediction problem (classification or regression) on a single instance at a time.

E.g. if you are doing spam detection on email, you will look at all the features associated with that email and classify it as spam or not. The aim of traditional ML is to come up with a class (spam or no-spam) or a single numerical score for that instance.

LTR solves a ranking problem on a list of items. The aim of LTR is to come up with optimal ordering of those items. As such, LTR doesn't care much about the exact score that each item gets, but cares more about the relative ordering among all the items.

The most common application of LTR is search engine ranking, but it's useful anywhere you need to produce a ranked list of items.

The training data for a LTR model consists of a list of items and a "ground truth" score for each of those items. For search engine ranking, this translates to a list of results for a query and a relevance rating for each of those results with respect to the query. The most common way used by major search engines to generate these relevance ratings is to ask human raters to rate results for a set of queries

Learning to rank algorithms have been applied in areas other than information retrieval:

- In machine translation for ranking a set of hypothesized translations
- In computational biology for ranking candidate 3-D structures in protein structure prediction problem
- In recommender systems for identifying a ranked list of related news articles to recommend to a user after he or she has read a current news article
- In software engineering, learning-to-rank methods have been used for fault localization kernel k-means clustering algorithm

UNIT - IV

Model Validation in Classification : Cross Validation - Holdout Method, K-Fold, Stratified K-Fold, Leave-One-Out Cross Validation. Bias-Variance tradeoff, Regularization , Overfitting, Underfitting. **Ensemble Methods:** Boosting, Bagging, Random Forest.

Supervised Machine Learning: Model Validation, a Step by Step Approach

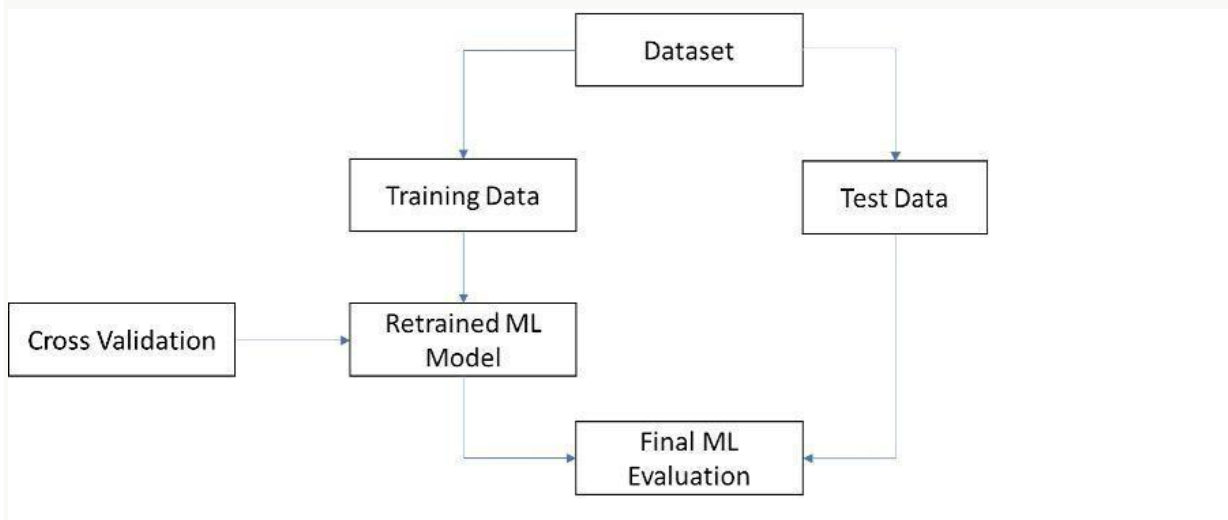
Model validation is the process of evaluating a trained model on test data set. This provides the generalization ability of a trained model. Here I provide a step by step approach to complete first iteration of model validation in minutes.

The basic recipe for applying a supervised machine learning model are:

1. Choose a class of model
2. Choose model hyper parameters
3. Fit the model to the training data
4. Use the model to predict labels for new data

What exactly is Cross-Validation?

CV is a technique used to train and evaluate an ML model using several portions of a dataset. This implies that rather than splitting the dataset into two parts only, one to train on and another to test on, the dataset is divided into more slices, or “folds”. And these slices use CV techniques to train the ML model so as to test its predictive capability and hence accuracy.



Cross-Validation Data Flow Overview

In the process of building a training set, different portions of data are gathered, while the remaining ones are reserved for constructing a validation set. This strategic approach ensures that the model continuously leverages new and diverse data during training and testing stages, promoting its ability to adapt to various scenarios and challenges.

One key objective of employing cross-validation is to safeguard the model against overfitting. Overfitting occurs when a model simply memorizes the samples in the training set, resulting in an artificially high predictive test score. However, such a model may struggle to generalize well on unseen data, leading to a lack of useful results. By validating the model's performance on a separate validation set, CV helps identify if the model has truly learned meaningful patterns and can generalize to new and unseen scenarios effectively.

The three key steps involved in CV are as follows:

1. Slice and reserve portions of the dataset for the training set,
2. Using what's left, test the ML model.
3. Use CV techniques to test the model using the reserve portions of the dataset created in step 1.

The Advantages of CV are as follows:

1. CV assists in realizing the optimal tuning of hyperparameters (or model settings) that increase the overall efficiency of the ML model's performance.
2. Training data is efficiently utilized as every observation is employed for both testing and training.

The Disadvantages of CV are as follows:

1. One of the main considerations with computer vision (CV) is the significant increase in testing and training time it requires for machine learning models. This is because CV involves multiple iterative testing cycles to ensure the accuracy and efficiency of the model. It includes various steps such as test preparation, execution, and rigorous analysis of the results to fine-tune and optimize the CV system. Therefore, understanding the time commitment involved in CV development is crucial for effectively leveraging its potential benefits.
2. Additional computation translates to increased resource demands. Cross Validation is known for its high computational expense, necessitating ample processing power. This results in the first drawback of extended time, which further inflates the budgetary requirements for an ML model project.

Two Types of Cross-Validation

Cross validation in machine learning is a crucial technique for evaluating the performance of predictive models. It involves dividing the available data into multiple subsets, or folds, to train and test the model iteratively. Non-exhaustive methods, such as k-fold cross-validation, randomly partition the data into k subsets and train the model on k-1 folds while evaluating it on the remaining fold. On the other hand, exhaustive methods, like leave-one-out cross-validation, systematically leave out one data point at a time for testing while training the model on the remaining data points. These methods provide a

comprehensive assessment of the model's performance and help in addressing overfitting or underfitting issues effectively.

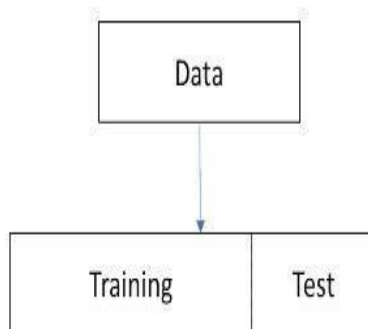
The five key types of CV in ML are:

1. Holdout Method
2. K-Fold CV
3. Stratified K-Fold CV
4. Leave-P-Out CV
5. Leave-One-Out CV

Holdout Method

The holdout method is a basic CV approach in which the original dataset is divided into two discrete segments:

1. Training Data - As a reminder this set is used to fit and train the model.
2. Test Data - This set is used to evaluate the model.



The Hold-out method splits the dataset into two portions

As a non-exhaustive method, the Hold-out model 'trains' the ML model on the training dataset and evaluates the ML model using the testing dataset.

In the majority of cases, the size of the training dataset is typically much larger than the test dataset. Therefore, a standard holdout method split ratio is 70:30 or 80:20. Furthermore, the overall dataset is randomly rearranged before dividing it into the training and test set portions using the predetermined ratio.

There are several disadvantages to the holdout method that need to be considered. One drawback is that as the model trains on distinct combinations of data points, it can sometimes yield inconsistent results, which can introduce doubt into the validity of the model and the overall validation process.

Another concern is that there is no certainty that the training dataset selected fully represents the

complete dataset. If the original data sample is not large enough, there is a possibility that the test data may contain information that the model will fail to recognize because it was not included in the original training data portion.

However, despite these limitations, the Holdout CV method can be considered ideal in situations where time is a scarce project resource and there is an urgency to train and test an ML model using a large dataset.

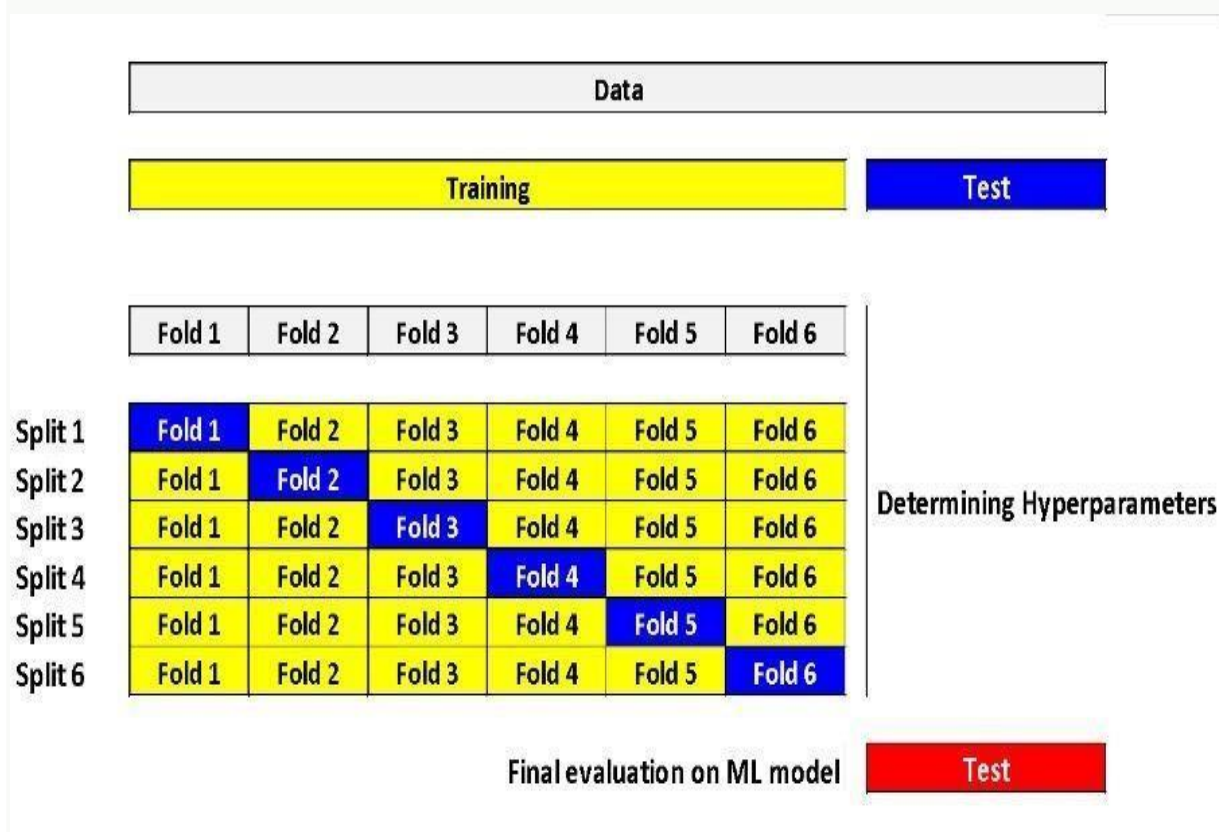
K fold Cross-Validation

The k-fold cross-validation method is considered an improvement over the holdout method due to its ability to provide additional consistency to the overall testing score of machine learning models. This improvement is achieved by applying a specific procedure for selecting and dividing the training and testing datasets.

To implement k-fold cross-validation, the original dataset is divided into k number of partitions. The holdout method is then performed k number of occasions, each time using a different partition as the testing set, while the remaining partitions are used for training. This repeated process helps to obtain a more reliable and robust evaluation of the model's performance by leveraging a larger amount of data for testing and training purposes.

Let us look at an example: if the value of **k** is set to six, there will be six subsets of equivalent sizes or folds of data. In the first iteration, the model trains on one subset and validates on the other. In the second iteration, the model re-trains on another subset and then is tested on the remaining subset. And so on for six iterations in total.

Diagrammatically this is shown as follows:



The k-fold cross-validation randomly splits the original dataset into k number of folds

The test results of each iteration are then averaged out, which is called the CV accuracy. Finally, CV accuracy is employed as a performance metric to contrast and compare the efficiencies of different ML models. It is important to note that the value of **k** is incidental or random. However, the **k** value is commonly set to ten within the data science field. The k-fold cross-validation approach is widely recognized for generating ML models with reduced subjectivity. By ensuring that each data point is present in both testing and training datasets, this technique enhances the objectivity of the models. Moreover, the k-fold method proves to be particularly advantageous for data science projects with a finite amount of data. It maximizes the utilization of available data by repeatedly utilizing different data sets.

Jake VanderPlas, gives the process of model validation in four simple and clear steps. There is also a whole process needed before we even get to his first step. Like fetching all the information we need from the data to make a good judgement for choosing a class model. Also providing finishing touches to confirm the results after. I will get into depth about these steps and break it down further.

- Data cleansing and wrangling.
- Split the data into training and test data sets.
- Define the metrics for which model is getting optimized.
- Get quick initial metrics estimate.
- Feature engineering to optimize the metrics. (*Skip this during first pass*).
- Data pre-processing.
- Feature selection.
- Model selection.
- Model validation.
- Interpret the results.
- Get the best model and check it against test data set.

Domain knowledge on the problem in hand will be of great use for feature engineering. This is a bigger topic in itself and requires extensive investment of time and resource.

Data pre-processing.

Data pre-processing converts features into format that is more suitable for the estimators. In general, machine learning models prefer standardization of the data set. I will make use of **RobustScaler** for our example.

Feature selection.

Feature selection or dimensionality reduction on data sets helps to

- Either to improve models' accuracy scores or
- To boost their performance on very high-dimensional data sets.

WHAT ARE ENSEMBLE MODELS?

Ensemble models are a machine learning approach to combine multiple other models in the prediction process. These models are referred to as base estimators. Ensemble models offer a solution to overcome the technical challenges of building a single estimator.\

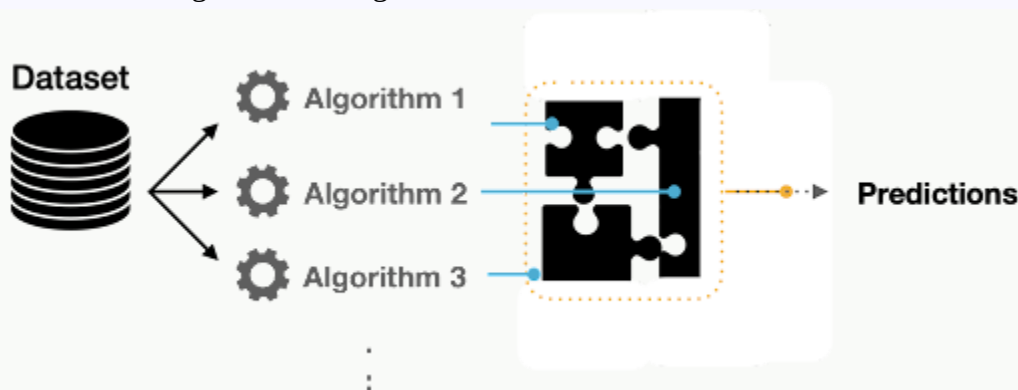
The technical challenges of building a single estimator include:

- **High variance:** The model is very sensitive to the provided inputs for the learned features.
- **Low accuracy:** One model (or one algorithm) to fit the entire training data might not provide you with the nuance your project requires.
- **Features noise and bias:** The model relies heavily on too few features while making a prediction.

Ensemble Algorithm

A single algorithm may not make the perfect prediction for a given data set. Machine learning algorithms have their limitations and producing a model with high accuracy is challenging. If we build and combine multiple models, we have the chance to boost the overall accuracy. We then implement the combination of models by aggregating the output from each model with two objectives:

1. Reducing the model error
2. Maintaining the model's generalization



TYPES OF ENSEMBLE MODELING TECHNIQUES

1. Bagging
2. Boosting
3. Stacking
4. Blending
- 5.

BAGGING

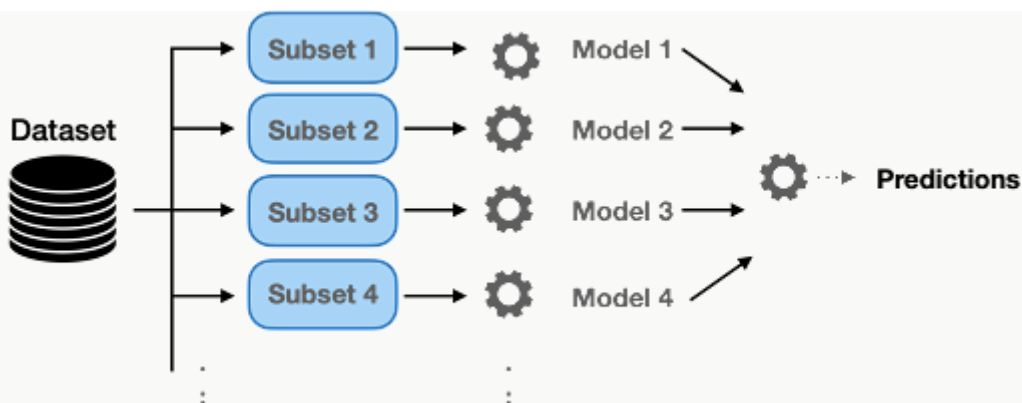
The idea of bagging is based on making the training data available to an iterative learning process. Each model learns the error produced by the previous model using a slightly different subset of the training data set. Bagging reduces variance and minimizes overfitting. One example of such a technique is the random forest algorithm.

Bootstrap Aggregation (Bagging)

This technique is based on a bootstrapping sampling technique. Bootstrapping creates multiple sets of the original training data with replacement. Replacement enables the duplication of sample instances in a set. Each subset has the same equal size and can be used to train models in parallel.

The method involves:

- Creating multiple subsets from the original dataset with replacement,
- Building a base model for each of the subsets,
- Running all the models in parallel,
- Combining predictions from all models to obtain final predictions.



Boosting

Boosting is a machine learning ensemble technique that reduces bias and variance by converting weak learners into strong learners. The weak learners are applied to the dataset in a sequential manner. The first step is building an initial model and fitting it into the training set.

A second model that tries to fix the errors generated by the first model is then fitted. Here's what the entire process looks like:

- Create a subset from the original data,
- Build an initial model with this data,
- Run predictions on the whole data set,
- Calculate the error using the predictions and the actual values,
- Assign more weight to the incorrect predictions,
- Create another model that attempts to fix errors from the last model,
- Run predictions on the entire dataset with the new model,
- Create several models with each model aiming at correcting the errors generated by the previous one,
- Obtain the final model by weighting the mean of all the models.

Random Forest Algorithm

Random Forest Algorithm widespread popularity stems from its user-friendly nature and adaptability, enabling it to tackle both classification and regression problems effectively. The algorithm's strength lies in its ability to handle complex datasets and mitigate overfitting, making it a valuable tool for various predictive tasks in machine learning.

One of the most important features of the Random Forest Algorithm is that it can handle the data set containing **continuous variables**, as in the case of regression, and **categorical variables**, as in the case of classification. It performs better for classification and regression tasks.

What is Random forest

A Random Forest is like a group decision-making team in machine learning. It combines the opinions of many "trees" (individual models) to make better predictions, creating a more robust and accurate overall model.

What is Random Forest Algorithm?

Random Forest Algorithm widespread popularity stems from its user-friendly nature and adaptability, enabling it to tackle both classification and regression problems effectively. The algorithm's strength lies in its ability to handle complex datasets and mitigate overfitting, making it a valuable tool for various predictive tasks in machine learning.

One of the most important features of the Random Forest Algorithm is that it can handle the data set containing ***continuous variables***, as in the case of regression, and ***categorical variables***, as in the case of classification. It performs better for classification and regression tasks. In this tutorial, we will understand the working of random forest and implement random forest on a classification task.

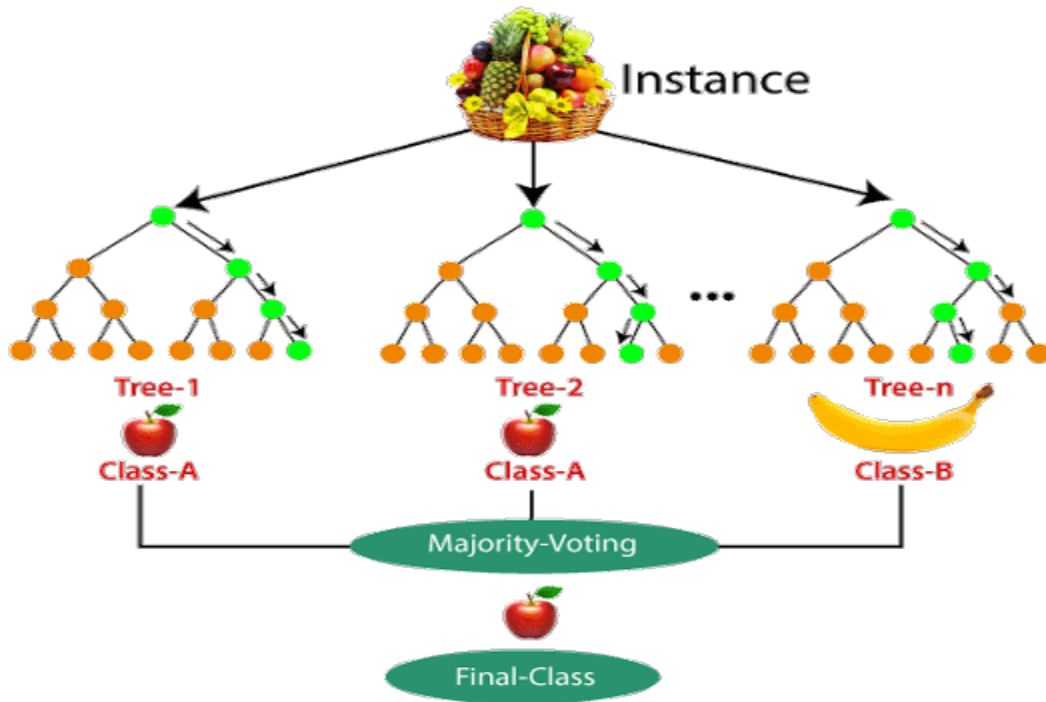
As mentioned earlier, Random forest works on the Bagging principle. Now let's dive in and understand bagging in detail.

Steps Involved in Random Forest Algorithm

- **Step 1:** In the Random forest model, a subset of data points and a subset of features is selected for constructing each decision tree. Simply put, n random records and m features are taken from the data set having k number of records.
- **Step 2:** Individual decision trees are constructed for each sample.
- **Step 3:** Each decision tree will generate an output.
- **Step 4:** Final output is considered based on ***Majority Voting or Averaging*** for Classification and regression, respectively.

For example

Consider the fruit basket as the data as shown in the figure below. Now n number of samples are taken from the fruit basket, and an individual decision tree is constructed for each sample. Each decision tree will generate an output, as shown in the figure. The final output is considered based on majority voting. In the below figure, you can see that the majority decision tree gives output as an apple when compared to a banana, so the final output is taken as an apple.



Important Features of Random Forest

- **Diversity:** Not all attributes/variables/features are considered while making an individual tree; each tree is different.
- **Immune to the curse of dimensionality:** Since each tree does not consider all the features, the feature space is reduced.
- **Parallelization:** Each tree is created independently out of different data and attributes. This means we can fully use the CPU to build random forests.
- **Train-Test split:** In a random forest, we don't have to segregate the data for train and test as there will always be 30% of the data which is not seen by the decision tree.
- **Stability:** Stability arises because the result is based on majority voting/ averaging.

UNIT – V

Unsupervised Learning : Clustering-K-means, K-Modes, K-Prototypes, Gaussian Mixture Models, Expectation-Maximization.

Reinforcement Learning: Exploration and exploitation trade-offs, non-associative learning, Markov decision processes, Q-learning.

Unsupervised Machine Learning:**Introduction to clustering**

As the name suggests, unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things. It can be defined as:

“Unsupervised learning is a type of machine learning in which models are trained using unlabeled dataset and are allowed to act on that data without any supervision.”

Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data. The goal of unsupervised learning is to **find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format**

Example: Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset. The task of the unsupervised learning algorithm is to identify the image features on their own. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.



Why use Unsupervised Learning?

Below are some main reasons which describe the importance of Unsupervised Learning:

- Unsupervised learning is helpful for finding useful insights from the data.
- Unsupervised learning is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.

Unsupervised learning works on unlabeled and uncategorized data which make unsupervised learning more important.

- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need unsupervised learning.

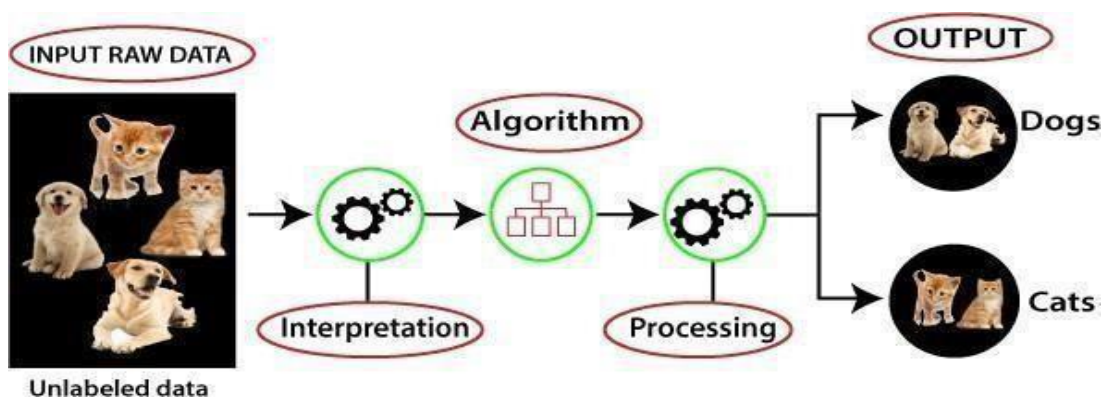
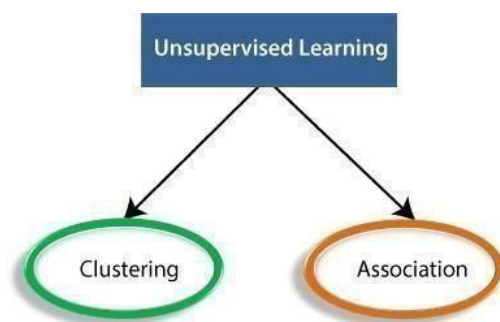
Working of Unsupervised Learning

we have taken an unlabeled input data, which means it is not categorized and corresponding outputs are also not given. Now, this unlabeled input data is fed to the machine learning model in order to train it. Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as k- means clustering, Decision tree, etc.

Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

Types of Unsupervised Learning Algorithm:

The unsupervised learning algorithm can be further categorized into two types of problems:



Clustering: Clustering is a method of grouping the objects into clusters such that objects with most similarities remain into a group and have less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.

- **Association:** An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.

Unsupervised Learning algorithms:

Below is the list of some popular unsupervised learning algorithms:

K-means clustering of unsupervised learning can be understood by the below diagram:

- KNN (k-nearest neighbors)
- Hierarchical clustering
- Anomaly detection
- Neural Networks
- Principle Component Analysis
- Independent Component Analysis
- Apriori algorithm
- Unsupervised learning is used for more complex tasks as compared to supervised learning because, in unsupervised learning, we don't have labeled input data.
- Unsupervised learning is preferable as it is easy to get unlabeled data in comparison to labeled data.

Disadvantages of Unsupervised Learning

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
- The result of the unsupervised learning algorithm might be less accurate as input data is not labeled, and algorithms do not know the exact output in advance.

k-means clustering algorithm

One of the most used clustering algorithm is *k-means*. It allows to group the data according to the existing similarities among them in *k* clusters, given as input to the algorithm. I'll start with a simple example.

Let's imagine we have 5 objects (say 5 people) and for each of them we know two features

(height and weight). We want to group them into $k=2$ clusters.

Our dataset will look like this:

How to apply k -means?

As you probably already know, I'm using Python libraries to analyze my data. The k -means algorithm is implemented in the *scikit-learn* package. To use it, you will just need the following line in your script:

What if our data is... non-numerical?

At this point, you will maybe have noticed something. The basic concept of k -means stands on mathematical calculations (means, euclidian distances). But what if our data is non-numerical or, in other words, *categorical*? Imagine, for instance, to have the ID code and date of birth of the five people of the previous example, instead of their heights and weights.

We could think of transforming our categorical values in numerical values and eventually apply k -means. But beware: k -means uses numerical distances, so it could consider close two really distant objects that merely have been assigned two close numbers.

k -modes is an extension of k -means. Instead of distances it uses *dissimilarities* (that is, quantification of the total mismatches between two objects: the smaller this number, the more similar the two objects). And instead of means, it uses *modes*. A mode is a vector of elements that minimizes the dissimilarities between the vector itself and each object of the data. We will have as many modes as the number of clusters we required, since they act as centroids.

K-means implements the **Expectation-Maximization** strategy to solve the problem. The Expectation-step is used to assign data points to the nearest cluster, and the Maximization-step is used to compute the centroid of each cluster.

When using the K-means algorithm, we must keep the following points in mind:

- It is suggested to normalize the data while dealing with clustering algorithms such as K-Means since such algorithms employ distance-based measurement to identify the similarity between data points.
- Because of the iterative nature of K-Means and the random initialization of centroids, K-Means may become stuck in a local optimum and fail to converge to the global optimum. As a result, it is advised to employ distinct centroids' initializations.

One of the conventional clustering methods commonly used in clustering techniques and efficiently used for large data is the K-Means algorithm. However, its method is not good and suitable for data that contains categorical variables. This problem happens when the cost function in K-Means is calculated using the Euclidian distance that is only suitable for numerical data. While K-Mode is only suitable for categorical data only, not mixed data types.

K-prototypes algorithm integrates the k-means and k-modes algorithms to deal with the mixed data types [2]. The k-prototypes algorithm is more useful practically because data collected in the real world are mixed type objects. Assume a set n objects, $X=\{X_1, X_2, \dots, X_n\}=\{1, 2, \dots, n\}$. $X_i=\{X_{i1}, X_{i2}, \dots, X_{im}\}=\{1, 2, \dots, \}$ consists of m attributes (m_r is numerical attributes, m_c is categorical attributes, $m=m_r+m_c$). The goal of clustering is to partition n objects into k disjoint clusters $C=\{C_1, C_2, \dots, C_k\}=\{1, 2, \dots, \}$, where C_i is an i -th cluster center. The distance $d(X_i, C_j)(\cdot)$ between X_i and C_j can be calculated as follows:

where $dr(Xi, Cj)(\cdot)$ is the distance between numerical attributes, $dc(Xi, Cj)(\cdot)$ is the distance between categorical attributes, and γ is a weight for categorical attributes.

$$dc(Xi,Cj)=\sum l=p+1m\delta(xil,cjl)(,)=\sum =+1(,)$$

(3)

In Equation (2), $dr(Xi, Cj)(.)$ is the squared Euclidean distance measure between cluster centers and an object on the numerical attributes. $dc(Xi, Cj)(.)$ is the simple matching dissimilarity measure on the categorical attributes, where $\delta(xil, cjl)(.) = 0$ for $xil=cjl$ and $\delta(xil, cjl)(.) = 1$ for $xil \neq cjl$. xil and cjl , $1 \leq l \leq p$, are values of numerical attributes, whereas xil and cjl , $p+1 \leq l \leq m+1$, are values of categorical attributes for object i and the cluster center j . p is the numbers of numerical attributes and $m-p$ is the numbers of categorical attributes.

Reinforcement learning

Reinforcement learning addresses the question of how an autonomous agent that senses and acts in its environment can learn to choose optimal actions to achieve its goals

Introduction

- Consider building a **learning robot**. The robot, or **agent**, has a set of sensors to observe the state of its environment, and a set of actions it can perform to alter this state.
- Its task is to learn a control strategy, or **policy**, for choosing actions that achieve its goals.
- The goals of the agent can be defined by a **reward function** that assigns a numerical value to each distinct action the agent may take from each distinct state.
- This reward function may be built into the robot, or known only to an external teacher who provides the reward value for each action performed by the robot.
- The **task** of the robot is to perform sequences of actions, observe their consequences, and learn a control policy.
- The control policy is one that, from any initial state, chooses actions that maximize the reward accumulated over time by the agent.

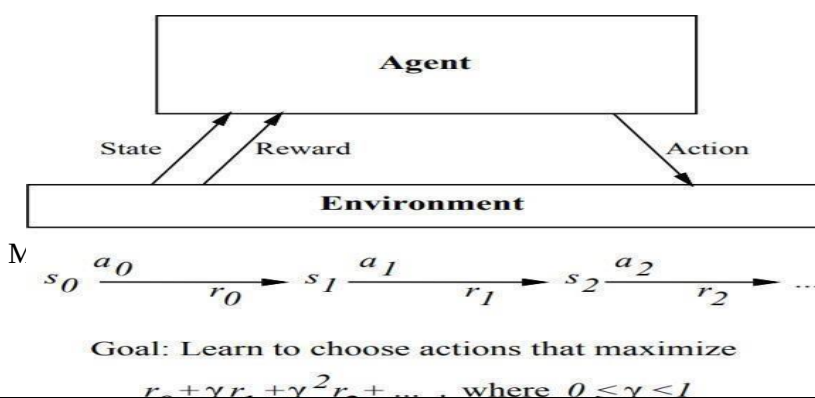
Example:

- A mobile robot may have sensors such as a camera and sonars, and actions such as "move forward" and "turn."
- The robot may have a goal of docking onto its battery charger whenever its battery level is low.
- The goal of docking to the battery charger can be captured by assigning a positive reward (Eg., +100) to state-action transitions that immediately result in a connection to the charger and a reward of zero to every other state-action transition.

Reinforcement Learning Problem

- An agent interacting with its environment. The agent exists in an environment described by some set of possible states S .
- Agent performs any of a set of possible actions A . Each time it performs an action a , in some state s the agent receives a real-valued reward r , that indicates the immediate value of this state-action transition. This produces a sequence of states s_i , actions a_i , and immediate rewards r_i as shown in the figure.

The agent's task is to learn a control policy, $\pi: S \rightarrow A$, that maximizes the expected sum of these rewards, with future rewards discounted exponentially by their delay.



Reinforcement learning problem characteristics

1. **Delayed reward:** The task of the agent is to learn a target function π that maps from the current state s to the optimal action $a = \pi(s)$. In reinforcement learning, training information is not available in $(s, \pi(s))$. Instead, the trainer provides only a sequence of immediate reward values as the agent executes its sequence of actions. The agent, therefore, faces the problem of **temporal credit assignment**: determining which of the actions in its sequence are to be credited with producing the eventual rewards.
2. **Exploration:** In reinforcement learning, the agent influences the distribution of training examples by the action sequence it chooses. This raises the question of which experimentation strategy produces most effective learning. The learner faces a trade-off in choosing whether to favor exploration of unknown states and actions, or exploitation of states and actions that it has already learned will yield high reward.
3. **Partially observable states:** The agent's sensors can perceive the entire state of the environment at each time step, in many practical situations sensors provide only partial information. In such cases, the agent needs to consider its previous observations together with its current sensor data when choosing actions, and the best policy may be one that chooses actions specifically to improve the observability of the environment.
4. **Life-long learning:** Robot requires to learn several related tasks within the same environment, using the same sensors. For example, a mobile robot may need to learn how to dock on its battery charger, how to navigate through narrow corridors, and how to pick up output from laser printers. This setting raises the possibility of using previously obtained experience or knowledge to reduce sample complexity when learning new tasks.

Learning Task

Consider Markov decision process (MDP) where the agent can perceive a set S of distinct states of its environment and has a set A of actions that it can perform

- At each discrete time step t , the agent senses the current state s_t , chooses a current action a_t , and performs it.
- The environment responds by giving the agent a reward $r_t = r(s_t, a_t)$ and by producing the succeeding state $s_{t+1} = \delta(s_t, a_t)$. Here the functions $\delta(s_t, a_t)$ and $r(s_t, a_t)$ depend only on the current state and action, and not on earlier states or actions.

The task of the agent is to learn a policy, $\pi: S \rightarrow A$, for selecting its next action a , based on the current observed state s_t ; that is, $\pi(s_t) = a_t$.

How shall we specify precisely which policy π we would like the agent to learn?

1. One approach is to require the policy that produces the greatest

possible **cumulative reward** for the robot overtime.

□ To state this requirement more precisely, define the cumulative value $V^\pi(s_t)$ achieved by following an arbitrary policy π from an arbitrary

$$\begin{aligned} V^\pi(s_t) &\equiv r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \\ &\equiv \sum_{i=0}^{\infty} \gamma^i r_{t+i} \end{aligned} \quad \text{equ (1)}$$

initial state s_t as follows:

□ Where, the sequence of rewards r_{t+i} is generated by beginning at state s_t and by repeatedly using the policy π to select actions.

□ Here $0 \leq \gamma \leq 1$ is a constant that determines the relative value of delayed versus immediate rewards. if we set γ

$= 0$, only the immediate reward is considered. As we set γ closer to 1, future rewards are given greater emphasis relative to the immediate reward.

□ The quantity $V^\pi(s_t)$ is called the **discounted cumulative reward** achieved by policy π from initial state s . It is reasonable to discount future rewards relative to immediate rewards because, in many cases, we prefer to obtain the reward sooner rather than later.

2. Other definitions of total reward is **finite horizon reward**,

$$\sum_{i=0}^h r_{t+i}$$

Considers the undiscounted sum of rewards over a finite number h of steps

3. Another approach is **average reward**

$$\lim_{h \rightarrow \infty} \frac{1}{h} \sum_{i=0}^h r_{t+i}$$

Considers the average reward per time step over the entire lifetime of the agent.

We require that the agent learn a policy π that maximizes $V^\pi(s)$ for all states s . such a policy is called an **optimal policy** and denote it by π^*

Refer the value function $V^{\pi^*}(s)$ an optimal policy as $V^*(s)$. $V^*(s)$ gives the maximum discounted cumulative reward that the agent can obtain starting from state s .

Example:

A simple grid-world environment is depicted in the diagram

□ The six grid squares in this diagram represent six possible states, or locations, for the agent.

□ Each arrow in the diagram represents a possible action the agent can take to move from one state to another.

[The number associated with each arrow represents the immediate reward $r(s, a)$ the agent receives if it executes the corresponding state-action transition

[The immediate reward in this environment is defined to be zero for all state-action transitions except for those leading into the state labelled G. The state G as the goal state, and the agent can receive reward by entering this state.

Once the states, actions, and immediate rewards are defined, choose a value for the discount factor γ , determine the optimal policy π^* and its value function $V^*(s)$.

Let's choose $\gamma = 0.9$. The diagram at the bottom of the figure shows one optimal

policy for this setting.

Values of $V^*(s)$ and $Q(s, a)$ follow from $r(s, a)$, and the discount factor $\gamma = 0.9$. An optimal policy, corresponding to actions with maximal Q values, is also shown.

The discounted future reward from the bottom centre state is

$$0 + \gamma 100 + \gamma^2 0 + \gamma^3 0 + \dots = 90$$

Non-Associative Learning:

Below is the dictionary definition of non-associative learning:

As applied to animal behavior, is instances where behavior toward stimulus changes in the absence of any apparent associated stimulus or event (such as a reward or punishment).

In non-associative learning, the person is being trained on how to respond to a certain situation. There is a right and a wrong answer.

Supervised learning algorithms use non-associative learning. These algorithms learn from the training data. Primarily, they are taught based on the assumption there is a right or wrong answer.

The cost function, or loss, associated with the algorithm, is a similar concept to 'punishment.'

In non-associative machine learning, you use the training data set to teach the machine learning algorithm how to predict on the data set.

This is instead of letting the algorithm learn for itself on what the outcome should be.

In other words, it represents the process of supervised machine learning.

1. REGRESSION ANALYSIS

The classic example of supervised ML using regression is the prediction of house prices.

For example, the number of rooms a house has (input) and the price of the house (output).

This training data will teach the machine how the number of rooms and price are related, allowing it to make predictions of the output, cost of a house, based on the inputs, number of rooms.

2. CLASSIFICATION ANALYSIS

If we move onto classification analysis, we begin to use machine learning to determine which group an object belongs to. One of the classic examples is whether or not a tumor is malignant or benign. Or you could use it to say yes or no if someone is likely to pass an exam.

Another example is, will this person develop diabetes? Yes or No.

In classification analysis, the labeled training data set will have a sample set of people and their characteristics alongside whether or not they developed diabetes.

This training data is there to teach the machine how different characteristics of a person's genetics or lifestyle contribute to whether or not they would get diabetes.

Q LEARNING

How can an agent learn an optimal policy π^ for an arbitrary environment?*

The training information available to the learner is the sequence of immediate rewards $r(s_i, a_i)$ for $i = 0, 1, 2, \dots$

Given this kind of training information it is easier to learn a numerical evaluation function defined over states and actions, then implement the optimal policy in terms of this evaluation function.

What evaluation function should the agent attempt to learn?

One obvious choice is V^* . The agent should prefer state s_1 over state s_2 whenever $V^*(s_1) > V^*(s_2)$, because the cumulative future reward will be greater from s_1 .

The optimal action in state s is the action a that maximizes the sum of the immediate reward $r(s, a)$ plus the value V^* of the immediate successor state, discounted by γ .

$$\pi^*(s) = \underset{a}{\operatorname{argmax}} [r(s, a) + \gamma V^*(\delta(s, a))] \quad \text{equ (3)}$$

The Q Function

The value of Evaluation function $Q(s, a)$ is the reward received immediately upon executing action a from state s , plus the value (discounted by γ) of

$$Q(s, a) \equiv r(s, a) + \gamma V^*(\delta(s, a)) \quad \text{equ (4)}$$

$$\pi^*(s) = \underset{a}{\operatorname{argmax}} Q(s, a) \quad \text{equ (5)}$$

Rewrite Equation (3) in terms of $Q(s, a)$ as

Equation (5) makes clear, it need only consider each available action a in its current state s and choose the action that maximizes $Q(s, a)$.

An Algorithm for Learning Q

- Learning the Q function corresponds to learning the **optimal policy**.
- The key problem is finding a reliable way to estimate training values for Q , given only a sequence of immediate rewards r spread out over

$$V^*(s) = \max_{a'} Q(s, a')$$

time. This can be accomplished through *iterative approximation*

$$Q(s, a) = r(s, a) + \gamma \max_{a'} Q(\delta(s, a), a')$$

Rewriting Equation

- Q learning algorithm:

Q learning algorithm

For each s, a initialize the table entry $\hat{Q}(s, a)$ to zero.

Observe the current state s

Do forever:

- Select an action a and execute it
- Receive immediate reward r
- Observe the new state s'
- Update the table entry for $\hat{Q}(s, a)$ as follows:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

- $s \leftarrow s'$
-

Q learning algorithm assuming deterministic rewards and actions.

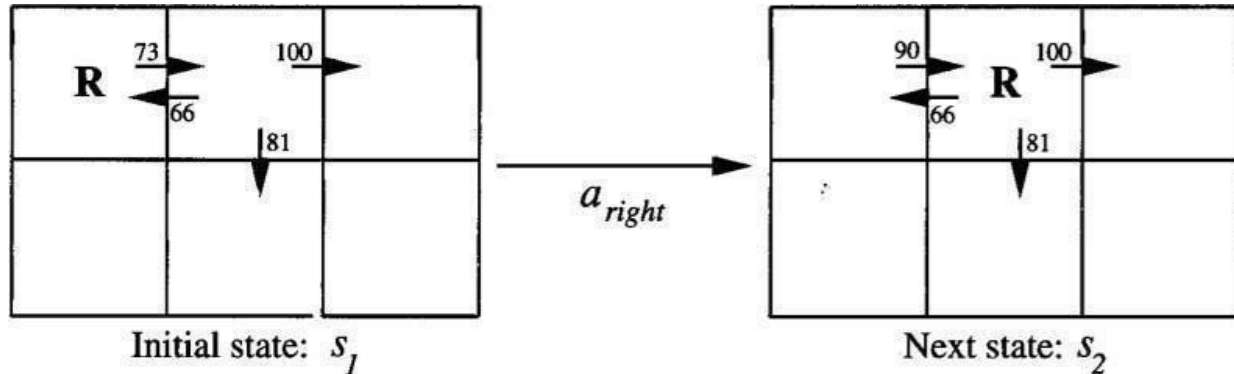
The discount factor γ may be any constant such that $0 \leq \gamma < 1$

□ Q to refer to the learner's estimate, or hypothesis, of the actual Q function

An Illustrative Example

□ To illustrate the operation of the Q learning algorithm, consider a single action taken by an agent, and the corresponding refinement to

Q shown in below figure



□ The agent moves one cell to the right in its grid world and receives an immediate reward of zero for this transition.

□ Apply the training rule of Equation

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

to refine its estimate Q for the state-action transition it just executed.

According to the training rule, the new Q estimate for this transition is the sum of the received reward (zero) and the highest

$$\begin{aligned} \hat{Q}(s_1, a_{right}) &\leftarrow r + \gamma \max_{a'} \hat{Q}(s_2, a') \\ &\leftarrow 0 + 0.9 \max\{66, 81, 100\} \\ &\leftarrow 90 \end{aligned}$$

Q value associated with the resulting state (100), discounted by γ (.9).

Convergence

Will the Q Learning Algorithm converge toward a Q equal to the true Q function?

Yes, under certain conditions.

1. Assume the system is a deterministic MDP.
2. Assume the immediate reward values are bounded; that is, there exists some positive constant c such that for all states s and actions a , $|r(s, a)| < c$
3. Assume the agent selects actions in such a fashion that it visits every possible state-action pair infinitely often

Here are four machine learning trends that could become a reality in the near future:

1) **Intelligence on the Cloud**

Algorithms can help companies unearth insights about their business, but this proposition can be expensive with no guarantees of a bottom-line increase. Companies often deal with having to collect data, hire data scientists and train them to deal with changing databases. Now that more data metrics are becoming available, the cost to store it is dropping thanks to the cloud. There will no longer be the need to manage infrastructure as cloud systems can generate new models as the scale of an operation increases, while also delivering more accurate results. More open-source ML frameworks are coming to the fold, obtaining pre-trained platforms that can tag images, recommend products and perform natural language processing tasks.

2) **Quantum Computing Capabilities**

Some of the tasks that ML can help companies deal with is the manipulation and classification of large quantities of vectors in high-dimensional spaces. Current algorithms take a large chunk of time to solve these problems, costing companies more to complete their business processes. Quantum computers are slated to become all the rage soon as they can manipulate high-dimensional vectors at a fraction of the time. These will be able to increase the number of vectors and dimensions that are processed when compared to traditional algorithms in a quicker period of time.

3) **Improved Personalization**

Retailers are already making waves in developing recommendation engines that reach their target audience more accurately. Taking this a step further, ML will be able to improve the personalization techniques of these engines in more precise ways. The technology will offer more specific data that they can then use on ads to improve the shopping experience for consumers.

4) **Data on Data**

As the amount of data available increases, the cost of storing this data decreases at roughly the same rate. ML has great potential in generating data of the highest quality that will lead to better models, an improved user experience and more data that helps repeat but improve upon this cycle. Companies such as Tesla add a million miles of driving data to enhance its self-driving capabilities every hour. Its Autopilot feature learns from this data and improves the software that propels these self-driving vehicles forward as the company gathers more data on the possible pitfalls of autonomous driving technology.

